

Verbeter je schaakspel met data: een persoonlijke case study

Eindwerk opleiding Data analist – Syntra

Goran Schittekatte

Inhoud

1. Introductie	3
2. ETL-flow.....	4
2.1. Feitentabel ‘Spellen’	4
2.1.1. Documentatie tabel ‘Spellen’	5
2.2. Dimensietabel ‘Speltypes’	21
2.2.1. Documentatie tabel ‘Speltypes’	21
2.3. Dimensietabel ‘Openingen’	22
2.3.1. Documentatie tabel ‘Openingen’	22
2.4. Dimensietabel ‘Spelers’	24
2.4.1. Documentatie tabel ‘Spelers’	24
2.5. Feitentabel ‘Mijnspellen’	28
2.5.1. Documentatie tabel ‘Mijnspellen’	28
2.6. Dimensietabel ‘Spelers’ opnieuw invullen	49
2.6.1. Documentatie tabel ‘Spelers’ opnieuw invullen.....	49
2.7. Extra’s.....	54
2.8. Einde ETL-flow.....	55
3. Analyse en Visualisatie	56
3.1. Data mart.....	56
3.2. Mijn schaakspel verbetert niet. Integendeel!.....	57
3.3. Hoe word ik een betere schaakspeler?	59
3.3.1. Betrek theorie in je schaakspel	59
3.3.1.1. Openingen voor zwart	59
3.3.1.2. Openingen voor wit	66
3.3.2. Analyseer je eigen spellen	72
4. Conclusies.....	81
4.1. Plan van aanpak	81
5. Bibliografie.....	82

1. Introductie

Een drietal jaar geleden raakte ik geïnspireerd door de serie ‘The Queen’s Gambit’ om te beginnen met schaken. Het voelde meteen aan als iets voor mij en ik maakte snel vooruitgang.

Al gauw installeerde ik de app ‘Lichess’ om te kunnen schaken op mijn smartphone. Daarop had ik een sterke start en ben ik zo hoog als 1300 ELO (= ranking) geraakt in de eerste maanden. Om een beeld te schetsen van deze rang: ELO varieert ongeveer tussen 0 en 3000. Je begint met 800 en afhankelijk van je overwinningen of nederlagen zal je ofwel stijgen, ofwel dalen. 1300 ELO vond ik m.a.w. al zeer goed voor mijn ervaringsniveau, zeker omdat ik ‘zomaar wat deed’. Dit vlakke uiteindelijk nog af naar een stabiele 1250 ELO na wat meer partijen als ik het mij goed herinner.

We zijn nu een kleine drie jaar later en ik schaak nog steeds bijna dagelijks. In de laatste jaren heb ik wat YouTube-videos over schaken bekeken, in een paar recreatieve toernooien gespeeld en een algemene know-how opgedaan van de schaakwereld. Jammer genoeg heb ik in die tijd niet actief gewerkt aan het verbeteren van mijn schaakspel, vooral omdat ik niet goed wist waar te beginnen.

Ik heb de laatste tijd het gevoel dat mijn schaakvaardigheden niet meer verbeterd zijn sinds die eerste maanden. Erger nog, ik denk eigenlijk dat het slechter gaat. Mijn ELO ligt nu meestal onder de 1200. Het verschil met vroeger is dus niet zo groot, maar je zou denken dat drie jaar lang bijna elke dag schaken het tegenovergestelde resultaat zou opleveren.

In dit eindwerk wil ik met behulp van data een betrouwbaar onderzoek uitvoeren over hoe ik een betere schaakspeler wordt.

Nog voor we aan de data beginnen, wil ik op zijn minst al een idee hebben over hoe een schaakspeler zijn vaardigheden kan aanscherpen. Er zijn namelijk tal van video’s, artikels en webpagina’s op het internet die dit onderwerp aankaarten.

Na een beetje ‘googelen’ kan ik het volgende afleiden:

Om een betere schaakspeler te worden, moet je ...

- Theorie betrekken in je spel. Meer specifiek: enkele openingen leren (Kim, 2024).
- Langere, tragere speltypes spelen (Silman, 2016).
- Achteraf je schaakpartijen analyseren (Lu, 2021), (Shah, 2016).
- Op regelmatige basis schaakpuzzels oplossen, liefst als opwarming voordat je aan een spel begint (GothamChess, 2020).
- Topspelers, alias ‘grootmeesters’ volgen en hun partijen bekijken (Studer, 2021)

Met deze zaken in het achterhoofd kan het onderzoek van start gaan. We beginnen met de verzameling van de data.

2. ETL-flow

In deze eerste fase van het onderzoek wordt beschreven:

- Waar de data vandaan komt
- Hoe ik de data geïmporteerd heb in mijn database
- Welke transformaties er ondernomen werden in de data
- Hoe ik de data opgeladen heb in Power BI

Mijn database is SQL Server en ik gebruik Visual Studio code als tool om er bewerkingen in te doen. De data mart wordt in Power BI gemaakt, dus **Power BI kan min of meer gezien worden als mijn data warehouse.**

De tools die gebruikt werden:

- **Python**
- **Powershell**
- **Visual Studio Code** (SQL Server)
- **Power BI** (in deze fase enkel het inladen van de tabellen uit SQL Server)
- **Stockfish** (de engine om schaakanalyses uit te voeren)

De tabellen die aangemaakt werden:

- **Spellen** (feit)
- **MijnSpellen** (feit)
- **Speltypes** (dimensie)
- **Openingen** (dimensie)
- **Spelers** (dimensie)

2.1. Feitentabel 'Spellen'

Ik wil eerst en vooral een grote algemene feitentabel hebben van een selectie schaakpartijen. Hierin wil ik zoveel mogelijk spellen zetten, zodat de analyses zeker betrouwbare resultaten zullen hebben. Deze tabel noem ik 'Spellen'.

Hiervoor kan ik beroep doen op de open database van het populaire schaakplatform Lichess (Lichess, s.j.).

Ik heb beslist om één miljoen partijen per jaar, van de laatste 10 jaar te importeren naar mijn tabel. Dit is goed voor 10 miljoen partijen/rijen in totaal. Dat klinkt misschien als veel, maar elke maand worden er bijna 100 miljoen partijen gespeeld. Ik neem slechts 1% uit één maand per jaar uit de Lichess database.

(Eerst had ik geprobeerd om te werken met 50 miljoen partijen, maar het hele extract-proces duurde enorm lang en later in de transformatiefase botste ik op allerlei limieten van de software en van mijn laptop.)

2.1.1. Documentatie tabel 'Spellen'

Ga naar de open database van Lichess.

Kies file format '.pgn'.

dit downloadt een soort zip-bestand (.zst-bestand).

Verplaats het gedownloadde zip bestand naar dit pad: 'C:\eindwerk'.

Nu het zip-bestand uitpakken met powershell:

- `cd C:\eindwerk`
- `.\zstd -d C:\eindwerk\bestandsnaam`

Het bestand is uitgepakt en je hebt een .pgn-bestand.

Dit .pgn-bestand is te groot om om te zetten naar .csv, dus eerst moet dit in kleinere .pgn files gesplitst worden.

Maak een tekstbestand aan en sla die op als 'split_schaak_pgn.ps1'

Zet hierin de volgende Powershell-code:

```
➤ $inputFile = "C:\eindwerk\bestandsnaam"

$outputFolder = "C:\Spellen_batches"

$gamesPerFile = 1000000 # Aantal games per batch (hier moet ik om de één of andere
reden het dubbele zetten van wat ik wil)

$fileCount = 1

$gameCount = 0

# Controleer of inputbestand bestaat
if (!(Test-Path $inputFile)) {
    Write-Host "Fout: Inputbestand niet gevonden!" -ForegroundColor Red
    exit
}

# Maak de output folder als deze nog niet bestaat
if (!(Test-Path $outputFolder)) {
    New-Item -ItemType Directory -Path $outputFolder
```

```

}

# Open input PGN bestand

$reader = [System.IO.StreamReader]::new($inputFile)
$outputFile = "$outputFolder\batch_$fileCount.pgn"
$writer = [System.IO.StreamWriter]::new($outputFile)

while (($line = $reader.ReadLine()) -ne $null) {
    $writer.WriteLine($line)

    # Elke partij eindigt met een lege regel
    if ($line -eq "") {
        $gameCount++
    }

    # Toon vooruitgang elke 100.000 games
    if ($gameCount % 100000 -eq 0 -and $gameCount -gt 0) {
        Write-Host "$gameCount games verwerkt in batch_$fileCount.pgn..."
    }

    # Als we de limiet van het aantal partijen per batch bereiken, een nieuw bestand
    # openen
    if ($gameCount -ge $gamesPerFile) {
        $writer.Close()

        Write-Host "Batch $fileCount opgeslagen als batch_$fileCount.pgn"

        # Stop met bestanden maken als er 2 zijn aangemaakt

```

```

    if ($fileCount -ge 2) {
        Write-Host "Maximale aantal batchbestanden (2) bereikt. Het proces is gestopt." -
ForegroundColor Yellow
        break
    }

    $fileCount++

    $gameCount = 0

    $outputFile = "$outputFolder\batch_$fileCount.pgn"

    $writer = [System.IO.StreamWriter]::new($outputFile)
}
}

# Sluit bestanden

$writer.Close()

$reader.Close()

Write-Host "Het Splitsen is gebeurd. De kleine .pgn-files zijn opgeslaan in
$outputFolder"

```

Nu de python code uitvoeren in powershell:

- Open powershell als administrator
- Geef jezelf de juiste machtiging: Set-ExecutionPolicy Unrestricted -Scope Process
- Y
- cd C:\eindwerk
- .\split_schaak_pgn.ps1

Er zouden 2 batches van 500.000 games gemaakt moeten zijn van deze .pgn-file (en zijn opgeslaan in 'C:\Spellen_batches').

Nu de twee pgn-batches omzetten naar .csv-files.

Maak een nieuwe map aan 'C:\Spellen_csv'.

importeer de benodigde python-pakketten:

➤ Pip install python-chess pandas pyodbc

Maak een tekstbestand aan 'convert_pgn_naar_csv.py' en sla het op in 'C:\eindwerk' met daarin deze python code:

```
➤ import chess.pgn
import pandas as pd
import os
```

```
pgn_folder = "C:\\Spellen_batches"
csv_folder = "C:\\Spellen_csv"
```

```
# Zorg ervoor dat de outputfolder bestaat
os.makedirs(csv_folder, exist_ok=True)
```

```
# Vaste lijst met gewenste headers
```

```
HEADERS = [
    "Event", "Site", "Date", "Round", "White", "Black", "Result", "UTCDate", "UTCTime",
    "WhiteElo", "BlackElo", "WhiteRatingDiff", "BlackRatingDiff", "WhiteTitle", "ECO",
    "Opening", "TimeControl", "Termination"
]
```

```
# Functie om PGN naar CSV om te zetten
```

```
def pgn_to_csv(pgn_file, csv_file, max_games=500000):
    with open(pgn_file, "r", encoding="utf-8") as f:
        games = []
        game_count = 0

        while True:
            game = chess.pgn.read_game(f)
            if game is None or game_count >= max_games:
                break

            headers = game.headers
            moves = game.board().variation_san(game.mainline_moves())
```



```

# Voeg alleen de gewenste headers toe
game_data = {header: headers.get(header, "") for header in HEADERS}
game_data["Moves"] = moves

games.append(game_data)
game_count += 1

if game_count % 100000 == 0:
    print(f" {game_count} games verwerkt...")

# Zet de lijst om naar een DataFrame en sla op als CSV
df = pd.DataFrame(games)
df.to_csv(csv_file, index=False)
print(f" {csv_file} opgeslagen!")

# Doorloop alle PGN-bestanden en zet ze om naar CSV
for pgn_file in os.listdir(pgn_folder):
    if pgn_file.endswith(".pgn"):
        pgn_path = os.path.join(pgn_folder, pgn_file)
        csv_path = os.path.join(csv_folder, pgn_file.replace(".pgn", ".csv"))
        print(f" bezig met {pgn_file}...")
        pgn_to_csv(pgn_path, csv_path)

print("Alle PGN-bestanden zijn omgezet naar CSV")

```

Nu de code uitvoeren in powershell:

- Open powershell als administrator
- Geef jezelf de juiste machtiging: Set-ExecutionPolicy Unrestricted -Scope Process
- Y
- cd C:\eindwerk
- python convert_pgn_naar_csv.py

Nu heb je twee .csv-files met elk 500.000 schaakspellen.

Deze 1.000.000 games al opladen in SQL (want als ik de volgende games ophaal, zullen de oude files overschreven worden en ik weet niet hoe ik dit kan omzeilen).

Maak eerst de database 'schaken' aan in SQL.

➤ Create database Schaken

Dan de tabel 'Spellen' aanmaken.

➤ Create table Spellen (

Event nvarchar(255),

Site nvarchar(255),

Date nvarchar(50), -- lukt anders niet om de data op te laden

Round nvarchar(50),

White nvarchar(255),

Black nvarchar(255),

Result nvarchar(10),

UTCDate date,

UTCTime time,

WhiteElo nvarchar(10), -- lukt anders niet om de data op te laden

BlackElo nvarchar(10), -- lukt anders niet om de data op te laden

WhiteRatingDiff nvarchar(10), -- lukt anders niet om de data op te laden

BlackRatingDiff nvarchar(10), -- lukt anders niet om de data op te laden

WhiteTitle nvarchar(50),

ECO nvarchar(10),

Opening nvarchar(255),

TimeControl nvarchar(50),

Termination nvarchar(255),

Moves nvarchar(MAX)

)

Dan in SQL toestemming geven om in mijn lokale mappen te gaan.

➤ Exec sp_configure 'show advanced options', 1;

Reconfigure;

exec sp_configure 'xp_cmdshell', 1;

Reconfigure;

Dan de twee csv's importeren in de tabel 'Spellen'

➤ declare @filepath nvarchar(255) = 'C:\Spellen_csv\Batch_';

-- eerste file

Exec('

bulk insert Spellen

from ''' + @filepath + '1.csv'

with (

format = 'csv',

fieldterminator = ',',

rowterminator = '\n',

firstrow = 2,

tablock

')

-- tweede file

Exec('

bulk insert Spellen

from ''' + @filepath + '2.csv'

with (

format = 'csv',

fieldterminator = ',',

rowterminator = '\n',

```
firstrow = 2,  
tablock  
' )
```

In de oudste jaren (2015 en 2016) zijn de waarden in de datumkolom niet ingevuld (er staat overal '?????.??.??'. Dus die waarden verander ik telkens nog naar het desbetreffende jaar:

➤ update Spellen

```
set date = 'het desbetreffende jaartal'
```

```
where date = '?????.??.??'
```

De gegevens van één jaar zijn nu geïmporteerd. Dit telkens opnieuw doen tot allen de jaren geïmporteerd zijn in de tabel 'Spellen'.

Daarna:

Elke rij van de tabel 'Spellen' uniek maken:

Eerst ID-kolom toevoegen.

➤ alter table spellen

```
add Spel_ID int identity(1,1) primary key
```

Daarna een nieuwe tabel maken met de ID-kolom vanvoor.

➤ create table dbo.Spellen2 (

```
    Spel_ID int identity(1,1) primary key,
```

```
    Event nvarchar(255) NULL,
```

```
    Site nvarchar(255) NULL,
```

```
    Date nvarchar(50) NULL,
```

```
    Round nvarchar(50) NULL,
```

```
    White nvarchar(255) NULL,
```

```
    Black nvarchar(255) NULL,
```

```

Result nvarchar(10) NULL,
UTCDate date NULL,
UTCTime time(3) NULL,
WhiteElo nvarchar(10) NULL,
BlackElo nvarchar(10) NULL,
WhiteRatingDiff nvarchar(10) NULL,
BlackRatingDiff nvarchar(10) NULL,
WhiteTitle nvarchar(50) NULL,
ECO nvarchar(10) NULL,
Opening nvarchar(255) NULL,
TimeControl nvarchar(50) NULL,
Termination nvarchar(255) NULL,
Moves nvarchar(max) NULL
)

```

De data overzetten naar de nieuwe tabel.

➤ insert into dbo.Spellen2

```

select
    Event,
    Site,
    Date,
    Round,
    White,
    Black,
    Result,
    UTCDate,
    UTCTime,
    WhiteElo,
    BlackElo,
    WhiteRatingDiff,
    BlackRatingDiff,
    WhiteTitle,
    ECO,

```

```
Opening,  
TimeControl,  
Termination,  
Moves  
from dbo.spellen
```

Dan de oude tabel verwijderen.

- drop table Spellen

Tenslotte de nieuwe tabel hernoemen naar de originele naam.

- exec sp_rename 'Spellen2', 'Spellen'

Overbodige kolommen verwijderen:

- alter table Spellen
drop column Event, Site, Round, UTCdate, UTCtime, WhiteTitle, Opening,
WhiteRatingDiff, BlackRatingDiff

Nederlandse namen geven aan de kolommen:

- exec sp_rename 'Spellen.Date', 'Jaar', 'COLUMN';
exec sp_rename 'Spellen.White', 'Speler_Wit', 'COLUMN';
exec sp_rename 'Spellen.Black', 'Speler_Zwart', 'COLUMN';
exec sp_rename 'Spellen.Result', 'Uitslag_Spel', 'COLUMN';
exec sp_rename 'Spellen.WhiteElo', 'ELO_Wit', 'COLUMN';
exec sp_rename 'Spellen.BlackElo', 'ELO_Zwart', 'COLUMN';
exec sp_rename 'Spellen.ECO', 'Opening_ID', 'COLUMN';
exec sp_rename 'Spellen.Timecontrol', 'Tijdslimiet', 'COLUMN';
exec sp_rename 'Spellen.Termination', 'Manier_Einde_Spel', 'COLUMN';
exec sp_rename 'Spellen.Moves', 'Zetten_Spel', 'COLUMN';

Kolommen transformeren:

Tijdslimiet staat genoteerd in seconden + seconden, maar standaard wordt dit in minuten + seconden genoteerd. Dit aanpassen.

- delete from Spellen
where Tijdslimiet not like '%+%'

- update Spellen

set Tijdslimiet =

cast(cast(substring(Tijdslimiet, 1, charindex('+', Tijdslimiet) - 1) as int) / 60 as
varchar) +

'+' +

substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet))

Spellen waarin ik voorkom verwijderen, want ik ga mijn spellen later nog in een aparte tabel toevoegen. Dan heb ik zeker geen overlappingen.

- delete from Spellen

where [Speler_Wit] = 'Goran455'

or [Speler_Zwart] = 'Goran455'

Kolommen toevoegen:

Kolom 'Speltype' toevoegen (Bullet, Blitz, Rapid, Classic)

Het speltype wordt bepaald o.b.v. de 'totale tijd' van een tijdslimiet.

- Bullet: Totale tijd minder dan 3 minuten per speler
- Blitz: Totale tijd tussen 3 en 10 minuten per speler
- Rapid: Totale tijd tussen 10 en 30 minuten per speler
- Classical: Totale tijd meer dan 30 minuten per speler

Berekening totale tijd:

Totale Tijd = Basistijd + (extra tijd per zet * 40)

(* 40 is omdat er wordt uitgegaan van 40 zetten per speler)

Kolom 'Speltype' toevoegen

- alter table Spellen

add SpelType varchar(50)

- update Spellen

```

set Speltype =

CASE

    WHEN cast(left(Tijdslimiet, CHARINDEX('+', Tijdslimiet) - 1) as int) * 60
        + cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet))
as int) < 180 THEN 'Bullet'

    WHEN cast(left(Tijdslimiet, charindex('+', Tijdslimiet) - 1) as int) * 60
        + cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet))
as int) < 600 THEN 'Blitz'

    WHEN cast(left(Tijdslimiet, charindex('+', Tijdslimiet) - 1) as int) * 60
        + cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet))
as int) < 1800 THEN 'Rapid'

    WHEN cast(left(Tijdslimiet, CHARINDEX('+', Tijdslimiet) - 1) as int) * 60
        + cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet))
as int) >= 1800 THEN 'Classical'

    ELSE NULL

END

where Tijdslimiet like '%+%'

```

Nu een speltype ID maken door Speltype samen te voegen met Tijdslimiet.

- alter table Spellen
 - add Speltype_ID varchar(50)
- Update Spellen
 - set Speltype_ID = Speltype + ' - ' + Tijdslimiet

Nog twee kolommen toevoegen aan tabel 'Spellen'.

Deze gegevens zal ik mogelijks gebruiken in mijn zoektocht naar schaakopeningen.

➤ alter table Spellen

add Eerste_zet varchar(10),

Eerste_4_zetten varchar(50)

➤ WHILE 1 = 1

BEGIN

WITH FirstMove AS (

SELECT

s.Spel_Id,

tokens.value AS Eerste_zet,

ROW_NUMBER() OVER (PARTITION BY s.Spel_Id ORDER BY (SELECT NULL)) AS rn

FROM (

SELECT TOP (100000) *

FROM Spellen

WHERE Eerste_zet IS NULL

ORDER BY Spel_Id

) s

CROSS APPLY (

SELECT value

FROM STRING_SPLIT(REPLACE(REPLACE(s.zetten_spel, CHAR(13), ' '), CHAR(10), ' '), ' ')

WHERE value NOT LIKE '%.'

) AS tokens

)

UPDATE s

SET s.Eerste_zet = fm.Eerste_zet

FROM Spellen s

JOIN FirstMove fm ON s.Spel_Id = fm.Spel_Id

```
WHERE fm.rn = 1;
```

```
-- Break when done
```

```
IF @@ROWCOUNT = 0
```

```
    BREAK;
```

```
END
```

➤ DELETE FROM Spellen

```
WHERE Eerste_zet IS NULL;
```

➤ WHILE 1 = 1

```
BEGIN
```

```
    ;WITH FirstFourMoves AS (
```

```
        SELECT
```

```
            s.Spel_Id,
```

```
            STRING_AGG(tokens.value, ' ') WITHIN GROUP (ORDER BY tokens.token_order) AS  
Eerste_4_zetten
```

```
        FROM (
```

```
            SELECT TOP (100000) *
```

```
            FROM Spellen
```

```
            WHERE Eerste_4_zetten IS NULL
```

```
            ORDER BY Spel_Id
```

```
        ) s
```

```
        CROSS APPLY (
```

```
            SELECT
```

```
                value,
```

```
                ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS token_order
```

```
            FROM STRING_SPLIT(REPLACE(REPLACE(s.zetten_spel, CHAR(13), ' '), CHAR(10), '  
' ), ' ')
```

```

        WHERE value NOT LIKE '%.'
    ) tokens
    WHERE tokens.token_order <= 4
    GROUP BY s.Spel_Id
)
UPDATE s
SET s.Eerste_4_zetten = fm.Eerste_4_zetten
FROM Spellen s
JOIN FirstFourMoves fm ON s.Spel_Id = fm.Spel_Id;

-- Exit loop if no rows were updated
IF @@ROWCOUNT = 0
    BREAK;
END

```

Nog een kolom toevoegen aan tabel 'MijnSpellen':

Ik weet nog niet of ik dit nodig zal hebben. Maar het lijkt mij alleszins handig om het totale aantal zetten al in mijn tabel te hebben (vb. om de verschillen tussen lange en korte partijen te bekijken of om het gemiddelde aantal zetten per spel te berekenen)

```

➤ Alter table Spellen
add Totaal_aantal_zetten int

```

```

➤ WHILE 1 = 1
BEGIN
    ;WITH Batch AS (
        SELECT TOP (10000)
            Spel_ID,
            zetten_spel
        FROM Spellen
    )

```

```

WHERE Totaal_aantal_zetten IS NULL

ORDER BY Spel_ID
)

UPDATE s

SET Totaal_aantal_zetten = (

    SELECT COUNT(*)

    FROM STRING_SPLIT(REPLACE(REPLACE(b.zetten_spel, CHAR(13), ' '), CHAR(10), '
'), ' ')

    WHERE value NOT LIKE '%.'

    AND value <> ''

)

FROM Spellen s

JOIN Batch b ON s.Spel_ID = b.Spel_ID;

IF @@ROWCOUNT = 0 BREAK;

END

```

De winnende kleur wil ik duidelijker zien:

```

➤ WHILE 1 = 1

BEGIN

    ;WITH Batch AS (

        SELECT TOP (10000) Spel_ID

        FROM Spellen

        WHERE uitslag_spel IN ('0-1', '1-0') OR uitslag_spel NOT IN ('Zwart_win', 'Wit_win',
'Gelijkspel')

        ORDER BY Spel_ID

    )

    UPDATE s

    SET uitslag_spel =

```

```

CASE
    WHEN s.uitslag_spel = '0-1' THEN 'Zwart_win'
    WHEN s.uitslag_spel = '1-0' THEN 'Wit_win'
    ELSE 'Gelijkspel'
END
FROM Spellen s
JOIN Batch b ON s.Spel_ID = b.Spel_ID;

IF @@ROWCOUNT = 0 BREAK;
END

```

2.2. Dimensietabel ‘Speltypes’

2.2.1. Documentatie tabel ‘Speltypes’

De tabel aanmaken.

- create table Speltypes (
 - Speltype_ID nvarchar(50),
 - Speltype nvarchar(50),
 - Tijdslimiet nvarchar(50))

Dan de tabel invullen.

- insert into Speltypes (Speltype_ID, Speltype, Tijdslimiet)
 - select distinct Speltype_ID, Speltype, Tijdslimiet
 - from Spellen
 - where Speltype_ID is not NULL

Dan nog de overbodige kolommen wissen in feitentabel ‘Spellen’.

- alter table Spellen
 - drop column Speltype, Tijdslimiet

2.3. Dimensietabel 'Openingen'

2.3.1. Documentatie tabel 'Openingen'

De tabel aanmaken.

- create table Openingen (
 Opening_ID nvarchar(10) primary key,
 Opening_Naam nvarchar(255),
 Opening_Kleur nvarchar(10));

Tabel invullen (enkel de opening_ID. De naam en kleur ergens anders vinden).

- insert into Openingen (Opening_ID)
 select distinct Opening_ID
 from Spellen
 where Opening_ID is not NULL

De namen van de openingen heb ik kunnen vinden op Kaggle (Ramponi, 2023). Die namen (csv-file) in SQL opladen. Eerst een tijdelijke tabel maken.

- create table TempOpeningen (
 Opening_ID nvarchar(10),
 Opening_Name nvarchar(255));

Dan opladen in de tijdelijke tabel.

- Bulk insert TempOpeningen
 from 'C:\Eindwerk\namen_openingen.csv'
 with (
 format = "csv",
 fieldterminator = ";",
 rowterminator = "\n",
 firstrow = 2,

tablock)

Eerst de namen nog wat opkuisen in de tijdelijke tabel.

- update TempOpeningen
set Opening_Name = replace(replace(Opening_Name, '', ''), ';;', '')
- UPDATE TempOpeningen
set Opening_ID = replace(Opening_ID, '', '')

Dan joinen aan de echte 'Openingen' tabel.

- update o
set o.Opening_Naam = t.Opening_Name
from Openingen o
join TempOpeningen t ON o.Opening_ID = t.Opening_ID

De tijdelijke tabel verwijderen.

- drop table TempOpeningen

Er staat nog een vraagteken tussen de opening ID's.

- delete from Openingen
where Opening_ID = '?'

Ik wil dat vraagteken ook uit de tabel 'Spellen' halen.

- *delete from Spellen
where Opening_ID = '?'*
- *alter table Openingen
drop column Opening_Kleur*

2.4. Dimensietabel 'Spelers'

2.4.1. Documentatie tabel 'Spelers'

De tabel aanmaken.

```
➤ create table Spelers (  
    Speler_ID int identity(1,1) primary key,  
    Naam_Speler varchar(100) not NULL,  
    Aantal_Spellen_Gespeeld int,  
    Aantal_Spellen_Gewonnen int,  
    ELO_Actueel INT,  
)
```

Dan alle unieke spelersnamen in de nieuwe tabel zetten.

```
➤ insert into Spelers (Naam_speler)  
select distinct speler_naam  
from (  
    select speler_zwart as speler_naam from Spellen  
    UNION  
    select speler_wit AS speler_naam from Spellen  
) as unieke_spelers
```

De spelersnamen in de feitentabel 'Spellen' vervangen met de Speler ID die in dimensietabel 'Spelers' te vinden is

```
➤ update Spellen  
set speler_zwart = spelers.Speler_ID  
from Spellen  
join Spelers on Spellen.speler_zwart = spelers.naam_speler
```


- update Spellen
 - set speler_wit = spelers.Speler_ID
 - from Spellen
 - join spelers on Spellen.speler_wit = spelers.naam_speler

- Exec sp_rename 'Spellen.speler_zwart', 'Speler_ID_zwart', 'COLUMN';
 - Exec sp_rename 'Spellen.speler_wit', 'Speler_ID_wit', 'COLUMN';

De andere kolommen van dimensietabel 'Spelers' invullen.

- with SpellenTellen as (
 - select Speler_ID, count(*) as spellen_gespeeld
 - from (
 - select Speler_ID_zwart AS Speler_ID from Spellen
 - UNION ALL
 - select Speler_ID_wit as Speler_ID from Spellen
 -) as Alle_Spellen
 - group by Speler_ID
 -)
 - update spelers
 - set aantal_spellen_gespeeld = st.spellen_gespeeld
 - from spelers
 - join SpellenTellen st on spelers.Speler_ID = st.Speler_ID

- with WinsTellen AS (
 - select Speler_ID, count(*) AS spellen_gewonnen
 - from (
 - select Speler_ID_zwart AS Speler_ID
 - from Spellen

```

    where uitslag_spel = '0-1'

    UNION ALL

    select Speler_ID_wit as Speler_ID
    from Spellen
    where uitslag_spel = '1-0'

) as AlleWins

group by Speler_ID

)

update spelers
set aantal_spellen_gewonnen = wt.spellen_gewonnen
from spelers
join WinsTellen wt on spelers.Speler_ID = wt.Speler_ID;

```

➤ with AllePartijen as (

```

    select

        Speler_ID_zwart as Speler_ID,

        Jaar,

        cast(ELO_Zwart as float) as elo

    from Spellen

    where isnumeric(ELO_Zwart) = 1

    UNION ALL

    select

        Speler_ID_wit AS Speler_ID,

        Jaar,

        cast(ELO_Wit as float) as elo

    from Spellen

    where isnumeric(ELO_Wit) = 1

),

```

```

LaatsteJaarPerSpeler as (
    select Speler_ID, max(Jaar) as Meest_Recente_Jaar
    from AllePartijen
    group by Speler_ID
),
RecenteELO as (
    select ap.Speler_ID, ap.ELO
    from AllePartijen ap
    join LaatsteJaarPerSpeler lp
        on ap.Speler_ID = lp.Speler_ID
        and ap.Jaar = lp.Meest_Recente_Jaar
),
GemELOPerSpeler AS (
    select Speler_ID, avg(ELO) AS Avg_ELO
    from RecenteELO
    group BY Speler_ID
)
update spelers
set elo_Actueel = aep.Avg_ELO
from spelers
join GemELOPerSpeler aep
    on spelers.Speler_ID = aep.Speler_ID

```

2.5. Feitentabel ‘Mijnspellen’

2.5.1. Documentatie tabel ‘Mijnspellen’

Nu mijn eigen schaakspellen inladen:

Eerst mijn spellen exporteren uit Lichess (Lichess, Lichess, s.j.).

De geëxporteerde file opslaan: “C:\Eindwerk\lichess_Goran455_2025-05-17.pgn”.

Maak een tekstbestand aan ‘convert_pgn_naar_csv_Mijnspellen.py’ en sla het op in ‘C:\eindwerk’ met daarin deze python code:

```
➤ import chess.pgn

import pandas as pd

# Input PGN file path
pgn_file = "Mijnspellen.pgn"

# List to store game data
games = []

# Open and read the PGN file
with open(pgn_file) as f:
    while True:
        game = chess.pgn.read_game(f)
        if game is None:
            break

        headers = game.headers

        moves = " ".join(str(move) for move in game.mainline_moves()) # Extract all moves

        # Convert date to proper format (YYYY-MM-DD)
```

```

date = headers.get("Date", "")

if date:
    try:
        # Convert date format to YYYY-MM-DD
        date = pd.to_datetime(date, errors='coerce').strftime('%Y-%m-%d')
    except Exception as e:
        print(f"Error converting date {date}: {e}")
        date = ""

# Replace empty fields with None (NULL)
def handle_empty(value):
    return value.strip() if isinstance(value, str) else value

games.append({
    "Event": handle_empty(headers.get("Event", "")),
    "Site": handle_empty(headers.get("Site", "")),
    "Date": date,
    "White": handle_empty(headers.get("White", "")),
    "Black": handle_empty(headers.get("Black", "")),
    "Result": handle_empty(headers.get("Result", "")),
    "Gameld": handle_empty(headers.get("Gameld", "")),
    "UTCDate": handle_empty(headers.get("UTCDate", "")),
    "UTCTime": handle_empty(headers.get("UTCTime", "")),
    "WhiteElo": handle_empty(headers.get("WhiteElo", "")),
    "BlackElo": handle_empty(headers.get("BlackElo", "")),
    "WhiteRatingDiff": handle_empty(headers.get("WhiteRatingDiff", "")),
    "BlackRatingDiff": handle_empty(headers.get("BlackRatingDiff", "")),
    "Variant": handle_empty(headers.get("Variant", "")),

```

```

"TimeControl": handle_empty(headers.get("TimeControl", "")),
"ECO": handle_empty(headers.get("ECO", "")),
"Opening": handle_empty(headers.get("Opening", "")),
"Termination": handle_empty(headers.get("Termination", "")),
"Annotator": handle_empty(headers.get("Annotator", "")),
"Moves": moves # Store moves in PGN format
})

```

```
# Convert to DataFrame
```

```
df = pd.DataFrame(games)
```

```
# Save to CSV file with proper encoding (UTF-8)
```

```
df.to_csv("Mijnspellen.csv", index=False, encoding="utf-8")
```

```
print("de csv file Mijnspellen.csv is klaar")
```

Nu de python code uitvoeren in powershell:

- Open powershell als administrator
- Geef jezelf de juiste machtiging: Set-ExecutionPolicy Unrestricted -Scope Process
- Y
- cd C:\eindwerk
- convert_pgn_naar_csv_Mijnspellen.py

Mijn schaakspellen zijn nu omgezet naar een csv-file.

De tabel aanmaken in SQL.

- create table MijnSpellen (

```
Event nvarchar(255),
```

```
Site nvarchar(255),
```

```
Date nvarchar(50), -- lukt anders niet om de data op te laden
```

```
White nvarchar(255),
```

```

Black nvarchar(255),
Result nvarchar(10),
Gameld nvarchar(50),
UTCDate nvarchar(50), -- lukt anders niet om de data op te laden
UTCTime nvarchar(50), -- lukt anders niet om de data op te laden
WhiteElo nvarchar(10),
BlackElo nvarchar(10),
WhiteRatingDiff nvarchar(10),
BlackRatingDiff nvarchar(10),
Variant nvarchar(50),
TimeControl nvarchar(50),
ECO nvarchar(10),
Opening nvarchar(255),
Termination nvarchar(255),
Annotator nvarchar(255),
Moves nvarchar(MAX)
)

```

De csv-file inladen in SQL.

➤ bulk insert MijnSpellen

```

    from 'C:\Eindwerk\MijnSpellen.csv'
with (
    format = 'csv',
    fieldterminator = ',',
    rowterminator = '\n',
    firstrow = 2,
    tablock
)

```

Overbodige kolommen verwijderen:

- alter table MijnSpellen
drop column Event, Site, UTCDate, UTCTime, Annotator, Variant, WhiteRatingDiff,
BlackRatingDiff, Opening;

Nederlandse namen geven aan de kolommen:

- exec sp_rename 'MijnSpellen.Date', 'Datum', 'COLUMN';
exec sp_rename 'MijnSpellen.White', 'Speler_Wit', 'COLUMN';
exec sp_rename 'MijnSpellen.Black', 'Speler_Zwart', 'COLUMN';
exec sp_rename 'MijnSpellen.Result', 'Uitslag_Spel', 'COLUMN';
exec sp_rename 'MijnSpellen.GameID', 'MijnSpel_ID', 'COLUMN';
exec sp_rename 'MijnSpellen.WhiteElo', 'ELO_Wit', 'COLUMN';
exec sp_rename 'MijnSpellen.BlackElo', 'ELO_Zwart', 'COLUMN';
exec sp_rename 'MijnSpellen.ECO', 'Opening_ID', 'COLUMN';
exec sp_rename 'MijnSpellen.Timecontrol', 'Tijdslimiet', 'COLUMN';
exec sp_rename 'MijnSpellen.Termination', 'Manier_Einde_Spel', 'COLUMN';
exec sp_rename 'MijnSpellen.Moves', 'Zetten_Spel', 'COLUMN';

Kolommen opkuisen:

Tijdslimiet staat genoteerd in seconden + seconden, maar standaard wordt dit in minuten + seconden genoteerd. Dit aanpassen.

- Delete from MijnSpellen
Where Tijdslimiet not like '%+%'
- update MijnSpellen
Set Tijdslimiet =

cast(cast(substring(Tijdslimiet, 1, charindex('+', Tijdslimiet) - 1) as int) / 60 as
varchar) +

'+' +

substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet));

Kolommen toevoegen:

Kolom 'Speltype' toevoegen (Bullet, Blitz, Rapid, Classic).

- alter table MijnSpellen
add SpelType varchar(50);

- update MijnSpellen
set Speltype =
CASE
when cast(left(Tijdslimiet, charindex('+', Tijdslimiet) - 1) as int) * 60
+ cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet)) as
int) < 180 then 'Bullet'
when cast(left(Tijdslimiet, charindex('+', Tijdslimiet) - 1) as int) * 60
+ cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet)) as
int) < 600 then 'Blitz'
when cast(left(Tijdslimiet, charindex('+', Tijdslimiet) - 1) as int) * 60
+ cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet)) as
int) < 1800 then 'Rapid'
when cast(left(Tijdslimiet, CHARINDEX('+', Tijdslimiet) - 1) as int) * 60
+ cast(substring(Tijdslimiet, charindex('+', Tijdslimiet) + 1, len(Tijdslimiet)) as
int) >= 1800 then 'Classical'
ELSE NULL
END
where Tijdslimiet like '%+%'

Nu een speltype ID maken door Speltype samen te voegen met Tijdslimiet.

- alter table MijnSpellen
add Speltype_ID varchar(50)

- update MijnSpellen
set Speltype_ID = Speltype + ' - ' + Tijdslimiet

- alter table MijnSpellen
drop column Speltype, Tijdslimiet

Ik wil de ID-kolom vanvoor zien:

Een nieuwe tabel aanmaken met de ID kolom vanvoor.

- Create table MijnSpellen2 (
MijnSpel_ID nvarchar(50) Primary Key,
Datum nvarchar(50) NULL,
Speler_Wit nvarchar(255) NULL,
Speler_Zwart nvarchar(255) NULL,
Uitslag_Spel nvarchar(10) NULL,
ELO_Wit nvarchar(10) NULL,
ELO_Zwart nvarchar(10) NULL,
Opening_ID nvarchar(10) NULL,
Manier_Einde_Spel nvarchar(255) NULL,
Zetten_Spel nvarchar(MAX) NULL,
Speltype_ID nvarchar(50) NULL,
)

De data overzetten naar de nieuwe tabel.

- insert into MijnSpellen2
Select
MijnSpel_ID,
Datum,
Speler_Wit,
Speler_Zwart,
Uitslag_Spel,
ELO_Wit,
ELO_Zwart,
Opening_ID,
Manier_Einde_Spel,

```
Zetten_Spel,  
Speltype_ID  
from MijnSpellen
```

Dan de oude tabel verwijderen.

- drop table MijnSpellen

Tenslotte de nieuwe tabel hernoemen naar de originele naam.

- exec sp_rename 'MijnSpellen2', 'MijnSpellen';

Nog een extra kolom toevoegen aan tabel 'MijnSpellen': Goran_Kleur.

Met de waarden: 'Goran Wit' / 'Goran Zwart'. Die heb ik nodig om mijn winrates te bekijken.

- alter table MijnSpellen
add Goran_Kleur nvarchar(20)
- update MijnSpellen
set Goran_Kleur =
CASE
when Speler_Wit = 'Goran455' then 'Goran Wit'
when Speler_Zwart = 'Goran455' then 'Goran Zwart'
ELSE NULL
END

Nog een extra kolom toevoegen aan tabel 'MijnSpellen': Goran_Uitslag_Spel.

Met de waarden: Win Goran / Verlies Goran / Gelijkspel. Die heb ik ook nodig om mijn winrates te bekijken.

- alter table MijnSpellen
add Goran_Uitslag_Spel nvarchar(20)

➤ update MijnSpellen

set Goran_Uitslag_Spel =

CASE

when Goran_Kleur = 'Goran Zwart' and Uitslag_Spel = '0-1' then 'Goran Win'

when Goran_Kleur = 'Goran Wit' and Uitslag_Spel = '1-0' then 'Goran Win'

when Goran_Kleur = 'Goran Zwart' and Uitslag_Spel = '1-0' then 'Goran Verlies'

when Goran_Kleur = 'Goran Wit' and Uitslag_Spel = '0-1' then 'Goran Verlies'

when Uitslag_Spel = '1/2-1/2' then 'Gelijkspel'

ELSE NULL

END

Nog twee kolommen toevoegen aan tabel 'MijnSpellen':

De queries die ik voor tabel 'Spellen' gebruikte voor dezelfde kolommen werkten niet op deze tabel omdat er fouten ingekropen zijn bij het inladen van de csv-file. In sommige rijen zal een komma te weinig gestaan hebben. Eerst dit oplossen:

➤ delete from MijnSpellen

where Manier_Einde_Spel not in ('Time forfeit', 'Normal', 'Abandoned')

Nu de kolommen toevoegen.

➤ alter table MijnSpellen

add Eerste_zet varchar(10),

Eerste_4_zetten varchar(50)

Nog een probleem: De notatie van schaakzetten is anders in de tabel 'MijnSpellen' dan in de tabel 'Spellen'. Het toont de startpositie + eindpositie ipv enkel de eindpositie. Daarnaast zien we ook niet welk schaakstuk gebruikt wordt. Gelukkig kunnen we a.d.h.v. de beginpositie het juiste schaakstuk vinden en toewijzen (N=paard / B=loper / Q=koningin / K=koning / R=toren / niks=pion).

➤ update MijnSpellen

set Eerste_zet =

```

CASE left(zetten_spel, 2)
  when 'g1' then 'N' + substring(zetten_spel, 3, 2)
  when 'b1' then 'N' + substring(zetten_spel, 3, 2)
  when 'f1' then 'B' + substring(zetten_spel, 3, 2)
  when 'c1' then 'B' + substring(zetten_spel, 3, 2)
  when 'd1' then 'Q' + substring(zetten_spel, 3, 2)
  when 'e1' then 'K' + substring(zetten_spel, 3, 2)
  when 'a1' then 'R' + substring(zetten_spel, 3, 2)
  when 'h1' then 'R' + substring(zetten_spel, 3, 2)
  ELSE substring(zetten_spel, 3, 2)
END
where Eerste_zet is NULL

```

➤ delete from MijnSpellen

where Eerste_zet IS NULL

➤ WITH TokenizedMoves AS (

```

SELECT
  Mijnspel_ID,
  value AS move,
  ROW_NUMBER() OVER (PARTITION BY Mijnspel_ID ORDER BY (SELECT NULL)) AS
move_num,
  LEFT(value, 2) AS from_sq,
  SUBSTRING(value, 3, 2) AS to_sq
FROM Mijnspellen

CROSS APPLY STRING_SPLIT(REPLACE(REPLACE(zetten_spel, CHAR(13), ''),
CHAR(10), ' '), ' ') AS token

WHERE LEN(value) >= 4 AND value NOT LIKE '%.%'
),

```

MappedMoves AS (

SELECT

Mijnspel_ID,

move_num,

CASE

-- Paard

WHEN from_sq IN ('g1', 'b1', 'g8', 'b8') THEN 'N' + to_sq

-- loper

WHEN from_sq IN ('f1', 'c1', 'f8', 'c8') THEN 'B' + to_sq

-- Toren

WHEN from_sq IN ('a1', 'h1', 'a8', 'h8') THEN 'R' + to_sq

-- Koningin

WHEN from_sq IN ('d1', 'd8') THEN 'Q' + to_sq

-- Koning

WHEN from_sq IN ('e1', 'e8') THEN 'K' + to_sq

ELSE to_sq

END AS algebraic_move

FROM TokenizedMoves

WHERE move_num <= 4

),

AggregatedMoves AS (

SELECT

Mijnspel_ID,

STRING_AGG(algebraic_move, ' ') WITHIN GROUP (ORDER BY move_num) AS
Eerste_4_zetten

FROM MappedMoves

GROUP BY Mijnspel_ID

)

```

UPDATE s
SET s.Eerste_4_zetten = m.Eerste_4_zetten
FROM Mijnspellen s
JOIN AggregatedMoves m ON s.Mijnspel_ID = m.Mijnspel_ID;

```

Nog een kolom toevoegen aan tabel 'Mijnspellen':

➤ Alter table Mijnspellen

Add Totaal_aantal_zetten int

➤ update Mijnspellen

```

set Totaal_aantal_zetten = (
    select count(*)
    from string_split(replace(replace(zetten_spel, char(13), ' '), char(10), ' '), ' ')
    where len(value) >= 4 and value not like '%.%'
)

```

De winnende kleur wil ik duidelijker zien:

➤ Update Mijnspellen

Set uitslag_spel =

```

CASE
    when uitslag_spel = '0-1' then 'Zwart_win'
    when uitslag_spel = '1-0' then 'Wit_win'
    else 'Gelijkspel'
END

```

Ik wil de schaakanalyse-engine 'Stockfish' (Stockfishchess, s.j.) eens loslaten op mijn schaakspellen (die in de feitentabel 'Mijnspellen' staan).

Eerst mijn spellen terug exporteren uit Lichess (Lichess, Lichess, s.j.).

De geëxporteerde file opslaan: "C:\Eindwerk\lichess_Goran455_2025-05-24.pgn".

Dan Stockfish downloaden (Stockfishchess, s.j.).

De gedownloade zipfile uitpakken en opslaan: "C:\Eindwerk\stockfish-windows-x86-64-avx2\stockfish-windows-x86-64-avx2.exe".

De nodige python pakketten installeren.

➤ pip install chess stockfish tqdm

Sla een tekstbestand op: "C:\Eindwerk\Blunder_en_BestMove_Stockfish.py".

Zet daarin de volgende python code:

➤ import chess.pgn

from stockfish import Stockfish

import csv

import os

Paden en parameters

STOCKFISH_PATH = r"C:\Eindwerk\stockfish-windows-x86-64-avx2\stockfish-windows-x86-64-avx2.exe"

PGN_PATH = r"C:\Eindwerk\lichess_Goran455_2025-05-24.pgn"

CSV_PATH = r"C:\Eindwerk\Analyse_MijnSpellen_Stockfish.csv"

EVAL_THRESHOLD_BLUNDER = 300 # centipawns threshold for blunder

MAX_GAME_LENGTH = 150 # optional: skip very long games

stockfish = Stockfish(STOCKFISH_PATH)

Csv-file maken als die nog niet bestaat

if not os.path.isfile(CSV_PATH):

with open(CSV_PATH, mode="w", newline="", encoding="utf-8") as file:


```

writer = csv.writer(file)

writer.writerow([
    "game_id", "white", "black", "result",
    "blunder_move", "blunder_delta", "blunder_fen",
    "best_move", "best_delta", "blunder_count"
])

# Functie om 1 partij te analyseren
def analyze_game(game, game_id):
    board = game.board()
    move_data = []
    move_number = 1
    all_moves = list(game.mainline_moves())

    if len(all_moves) > MAX_GAME_LENGTH:
        print(f"Skipping Game {game_id} (too long)")
        return None

    for move in all_moves:
        try:
            fen_before = board.fen()
            stockfish.set_fen_position(fen_before)
            eval_before = stockfish.get_evaluation()

            board.push(move)

            stockfish.set_fen_position(board.fen())
            eval_after = stockfish.get_evaluation()

```

```

if eval_before["type"] == "cp" and eval_after["type"] == "cp":
    delta = eval_after["value"] - eval_before["value"]
    move_data.append({
        "move_number": move_number,
        "move": move.uci(),
        "delta": delta,
        "fen_before": fen_before
    })

    move_number += 1
except Exception as e:
    print(f" Evaluation error in Game {game_id}, Move {move_number}: {e}")
    board.push(move)
    move_number += 1
    continue

if not move_data:
    return None

biggest_blunder = min(move_data, key=lambda x: x["delta"])
best_move = max(move_data, key=lambda x: x["delta"])
all_blunders = [m for m in move_data if m["delta"] < -EVAL_THRESHOLD_BLUNDER]

return {
    "game_id": game_id,
    "white": game.headers.get("White", ""),
    "black": game.headers.get("Black", ""),

```

```

"result": game.headers.get("Result", ""),
"blunder_move": f"{biggest_blunder['move_number']}-{biggest_blunder['move']}",
"blunder_delta": biggest_blunder["delta"],
"blunder_fen": biggest_blunder["fen_before"],
"best_move": f"{best_move['move_number']}-{best_move['move']}",
"best_delta": best_move["delta"],
"blunder_count": len(all_blunders)
}

```

De loop

```

with open(PGN_PATH, encoding="utf-8") as pgn_file, \
    open(CSV_PATH, mode="a", newline="", encoding="utf-8") as csv_file:

```

```

    writer = csv.writer(csv_file)

```

```

while True:

```

```

    game = chess.pgn.read_game(pgn_file)

```

```

    if game is None:

```

```

        break

```

```

    game_id = game.headers.get("Gameld", "").strip()

```

```

    if not game_id:

```

```

        print("Warning: Game without Gameld found, skipping.")

```

```

        continue

```

```

    print(f"Analyzing Game {game_id}: {game.headers.get('White', '')} vs
{game.headers.get('Black', '')}")

```

```

    result = analyze_game(game, game_id)

```

```

if result:

    writer.writerow([

        result["game_id"], result["white"], result["black"], result["result"],

        result["blunder_move"], result["blunder_delta"], result["blunder_fen"],

        result["best_move"], result["best_delta"], result["blunder_count"]

    ])

    print("opgeslaan in de csv-file\n")

else:

    print("Overgeslaan (geen bruikbare data)\n")

```

Run dit script in powershell.

- cd C: \eindwerk
- python Blunder_en_BestMove_Stockfish.py

Nu zijn mijn grootste blunder en beste zet per spel + aantal blunders per spel + etc. opgeslaan in een csv-file ("C:\Eindwerk\Analyse_MijnSpellen_Stockfish.csv").

Dit nog inladen in SQL en dan joinen aan de 'Mijnspellen' tabel:

Eerst een tijdelijke tabel maken om de stockfish data in te kunnen laden.

- create table Analyse_Mijnspellen_Stockfish (

game_id nvarchar(20),

white nvarchar(100),

black nvarchar(100),

result nvarchar(10),

blunder_move nvarchar(20),

blunder_delta int,

blunder_fen nvarchar(MAX),

best_move nvarchar(20),

```
best_delta int,  
blunder_count int  
)
```

Daarna inladen in de tijdelijke tabel.

```
➤ bulk insert Analyse_Mijnspellen_Stockfish  
FROM 'C:\Eindwerk\Analyse_MijnSpellen_Stockfish.csv'  
with (  
    format = 'csv',  
    fieldterminator = ',',  
    rowterminator = '\n',  
    firstrow = 2,  
    tablock  
)
```

De gewenste kolommen toevoegen aan de 'MijnSpellen'-tabel.

```
➤ alter table Mijnspellen  
add  
    blunder_move nvarchar(20),  
    blunder_delta int,  
    blunder_fen nvarchar(MAX),  
    best_move nvarchar(20),  
    best_delta int,  
    blunder_count int
```

De Stockfish-data joinen aan de 'Mijnspellen'-tabel.

```
➤ update m  
set
```

```

m.blunder_move = a.blunder_move,
m.blunder_delta = a.blunder_delta,
m.blunder_fen = a.blunder_fen,
m.best_move = a.best_move,
m.best_delta = a.best_delta,
m.blunder_count = a.blunder_count
from Mijnspellen m
inner join Analyse_MijnSpellen_Stockfish a
    on m.MijnSpel_ID = a.game_id

```

➤ Delete from MijnSpellen

Where blunder_move is NULL

De tijdelijke Stockfish-tabel heb ik niet meer nodig.

➤ drop table Analyse_MijnSpellen_Stockfish

Nederlandse namen geven aan de kolommen.

```

➤ exec sp_rename 'mijnspellen.blunder_move', 'Blunder_zet', 'COLUMN';
exec sp_rename 'mijnspellen.blunder_delta', 'Blunder_puntenverlies', 'COLUMN';
exec sp_rename 'mijnspellen.blunder_fen', 'Laatste_positie_voor_blunder', 'COLUMN';
exec sp_rename 'mijnspellen.best_move', 'Beste_zet', 'COLUMN';
exec sp_rename 'mijnspellen.best_delta', 'Beste_puntenwinst', 'COLUMN';
exec sp_rename 'mijnspellen.blunder_count', 'Aantal_blunders', 'COLUMN';

```

De waarden van puntenverlies/puntenwinst uit de Stockfish analyse is in 'centipawns' uitgedrukt (vb. '-100' = een verlies ter waarde van 1 pion).

Ik wil dat dit in gewone punten uitgedrukt wordt (vb. '3' = een winst ter waarde van 3 pionnen of 1 paard of 1 looper)

➤ update MijnSpellen

set

Blunder_puntenverlies = cast(Blunder_puntenverlies as float) / 100,

Beste_puntenwinst = cast(Beste_puntenwinst as float) / 100

➤ alter table MijnSpellen

alter column Blunder_puntenverlies decimal(10,2)

➤ alter table MijnSpellen

alter column Beste_puntenwinst decimal(10,2)

Nog een kolom toevoegen in de 'Mijnspellen'-tabel: Ik wil weten welke ELO-waarde van mij is in elke rij. Dan kan ik daar berekeningen op doen om bvb. het bereik te bepalen waarbinnen mijn ELO meestal ligt.

➤ delete from MijnSpellen

Where (try_cast(ELO_Wit as int) is NULL and ELO_Wit is not NULL)

or (try_cast(ELO_Zwart as int) is NULL and ELO_Zwart is not NULL)

➤ alter table MijnSpellen

alter column ELO_Wit int

➤ alter table MijnSpellen

alter column ELO_Zwart int

(het datatype veranderen doe ik van de eerste keer ook in de tabel 'Spellen')

➤ delete from Spellen

where (try_cast(ELO_Wit as int) is NULL and ELO_Wit is not NULL)
or (try_cast(ELO_Zwart as int) is NULL and ELO_Zwart is not NULL)

➤ *alter table Spellen*
alter column ELO_Wit int

➤ *alter table Spellen*
alter column ELO_Zwart int

Nog een kolom toevoegen aan tabel 'Mijnspellen':

➤ *alter table MijnSpellen*
add ELO_Goran int

De Kolom opvullen.

➤ *update m*
set ELO_Goran =
CASE
when m.Speler_ID_Wit = s.Speler_ID then m.ELO_Wit
when m.Speler_ID_Zwart = s.Speler_ID then m.ELO_Zwart
ELSE NULL
END
from MijnSpellen m
inner join Spelers s
on s.Speler_ID = m.Speler_ID_Wit OR s.Speler_ID = m.Speler_ID_Zwart
where s.Naam_Speler = 'Goran455'

2.6. Dimensietabel 'Spelers' opnieuw invullen

Ik wil de 'spelers' tabel opnieuw invullen. Nu wil ik die vullen met de data uit de twee tabellen 'Spellen' en 'Mijnspellen' samen. (Dit ga ik niet doen voor de tabellen 'Openingen' en 'Speltypes', want dat zal geen verschil maken. Al de mogelijkheden zitten daar al in.)

2.6.1. Documentatie tabel 'Spelers' opnieuw invullen

➤ Update Spellen

```
set Speler_ID_zwart = s.naam_speler
```

```
From Spellen sp
```

```
Join spelers s on sp.Speler_ID_zwart = s.Speler_ID
```

➤ Update Spellen

```
set Speler_ID_wit = s.naam_speler
```

```
from Spellen sp
```

```
join spelers s ON sp.Speler_ID_wit = s.Speler_ID
```

➤ Exec sp_rename 'Spellen.Speler_ID_zwart', 'speler_zwart', 'COLUMN';

```
Exec sp_rename 'Spellen.Speler_ID_wit', 'speler_wit', 'COLUMN';
```

➤ select (*)

```
into spelers_backup
```

```
from spelers
```

➤ truncate table Spelers

➤ insert into Spelers (naam_speler)

```
select distinct speler_zwart from Spellen
```

```
UNION
```

```
select distinct speler_wit from Spellen
```

UNION

select distinct speler_zwart from MijnSpellen

UNION

select distinct speler_wit from MijnSpellen

➤ update spellen

set speler_zwart = s.Speler_ID

from spellen sp

join spelers s on sp.speler_zwart = s.naam_speler

➤ update Spellen

set speler_wit = s.Speler_ID

from Spellen sp

join Spelers s on sp.speler_wit = s.naam_speler

➤ update MijnSpellen

set speler_zwart = s.Speler_ID

from MijnSpellen ms

join Spelers s on ms.speler_zwart = s.naam_speler

➤ update MijnSpellen

set speler_wit = s.Speler_ID

from mijns spellen ms

join spelers s on ms.speler_wit = s.naam_speler

➤ Exec sp_rename 'spellen.speler_zwart', 'Speler_ID_zwart', 'COLUMN';

Exec sp_rename 'spellen.speler_wit', 'Speler_ID_wit', 'COLUMN';

➤ Exec sp_rename 'mijnspellen.speler_zwart', 'Speler_ID_zwart', 'COLUMN';

Exec sp_rename 'mijnspellen.speler_wit', 'Speler_ID_wit', 'COLUMN';

➤ with AantalSpellen as (

select Speler_ID, count(*) as spellen_gespeeld

from (

select Speler_ID_zwart AS Speler_ID from spellen

UNION ALL

select Speler_ID_wit from spellen

UNION ALL

select Speler_ID_zwart from MijnSpellen

UNION ALL

select Speler_ID_wit from MijnSpellen

) as Alle_Spellen

group by Speler_ID

)

update spelers

set aantal_spellen_gespeeld = gc.spellen_gespeeld

from spelers

join AantalSpellen gc on spelers.Speler_ID = gc.Speler_ID

➤ with WinsGeteld as (

select Speler_ID, count(*) as spellen_gewonnen

from (

select Speler_ID_zwart as Speler_ID from spellen where uitslag_speel = '0-1'

UNION ALL

select Speler_ID_wit from spellen where uitslag_speel = '1-0'

UNION ALL

```

select Speler_ID_zwart from mijnspelellen where uitslag_spele = '0-1'

UNION ALL

select Speler_ID_wit from mijnspelellen where uitslag_spele = '1-0'

) as AlleWins

group by Speler_ID

)

update Spelers

set aantal_spelellen_gewonnen = wg.spelellen_gewonnen

from Spelers

join WinsGeteld wg on spelers.Spelel_ID = wg.Spelel_ID

```

```

➤ WITH AllGames AS (

SELECT Spelel_ID_zwart AS Spelel_ID, Jaar, CAST(ELO_Zwart AS FLOAT) AS ELO

FROM Spelellen

WHERE ISNUMERIC(ELO_Zwart) = 1

```

```

UNION ALL

```

```

SELECT Spelel_ID_wit, Jaar, CAST(ELO_Wit AS FLOAT)

FROM Spelellen

WHERE ISNUMERIC(ELO_Wit) = 1

```

```

UNION ALL

```

```

SELECT Spelel_ID_zwart, YEAR(Datum) AS Jaar, CAST(ELO_Zwart AS FLOAT)

FROM Mijnspelellen

WHERE ISNUMERIC(ELO_Zwart) = 1

```

```

UNION ALL

SELECT Speler_ID_wit, YEAR(Datum), CAST(ELO_Wit AS FLOAT)
FROM MijnSpellen
WHERE ISNUMERIC(ELO_Wit) = 1
),
LatestYear AS (
    SELECT Speler_ID, MAX(Jaar) AS Most_Recent_Year
    FROM AllGames
    GROUP BY Speler_ID
),
RecentELOs AS (
    SELECT ag.Speler_ID, ag.ELO
    FROM AllGames ag
    JOIN LatestYear ly
    ON ag.Speler_ID = ly.Speler_ID
    AND ag.Jaar = ly.Most_Recent_Year
),
AvgELO AS (
    SELECT Speler_ID, AVG(ELO) AS Avg_ELO
    FROM RecentELOs
    GROUP BY Speler_ID
)
UPDATE spelers
SET ELO_Actueel = ae.Avg_ELO
FROM spelers
JOIN AvgELO ae ON spelers.Speler_ID = ae.Speler_ID;

```

```
UPDATE spelers
```

```
SET aantal_spellen_gewonnen = 0
```

```
WHERE aantal_spellen_gewonnen IS NULL;
```

➤ DROP TABLE spelers_backup

De tabel 'Spelers' is nu opnieuw ingevuld.

2.7. Extra's

Hier is nog de query die ik gebruikt heb om tussentijds een backup-file op te slaan van mijn database 'Schaken'.

➤ BACKUP DATABASE [schaken]

TO DISK = N'C:\Eindwerk\schaken_backup.bak'

WITH FORMAT,

MEDIA NAME = 'SchakenBackup',

NAME = 'Full Backup of schaken';

2.8. Einde ETL-flow

Tenslotte wordt de data die klaar staat in SQL Server opgeladen in Power BI:

➤ Connectie met SQL → server = 'localhost' → database = 'Schaken' → importeren.

Tijdens de ETL-fase heb ik reeds een aardige opkuis gedaan in de data. Daarnaast zijn er aan de tabellen al enkele berekende kolommen toegevoegd.

De berekeningen waar ik nu nog niet aan gedacht heb, voeg ik later wel toe in Power BI. Om die reden heb ik geen SQL-views gemaakt. Het leek mij efficiënter om meteen over te gaan naar de connectie tussen SQL en Power BI.

Nu dat achter de rug is, moet enkel de data mart nog gemaakt worden. Daarna kan ik beginnen aan de analyse en visualisatie in Power BI.

3. Analyse en Visualisatie

3.1. Data mart

Voordat de gegevens geanalyseerd kunnen worden, moet ik ervoor zorgen dat de tabellen gelinkt zijn aan elkaar o.b.v. de juiste soort relaties, zodat ze vlot samenwerken.

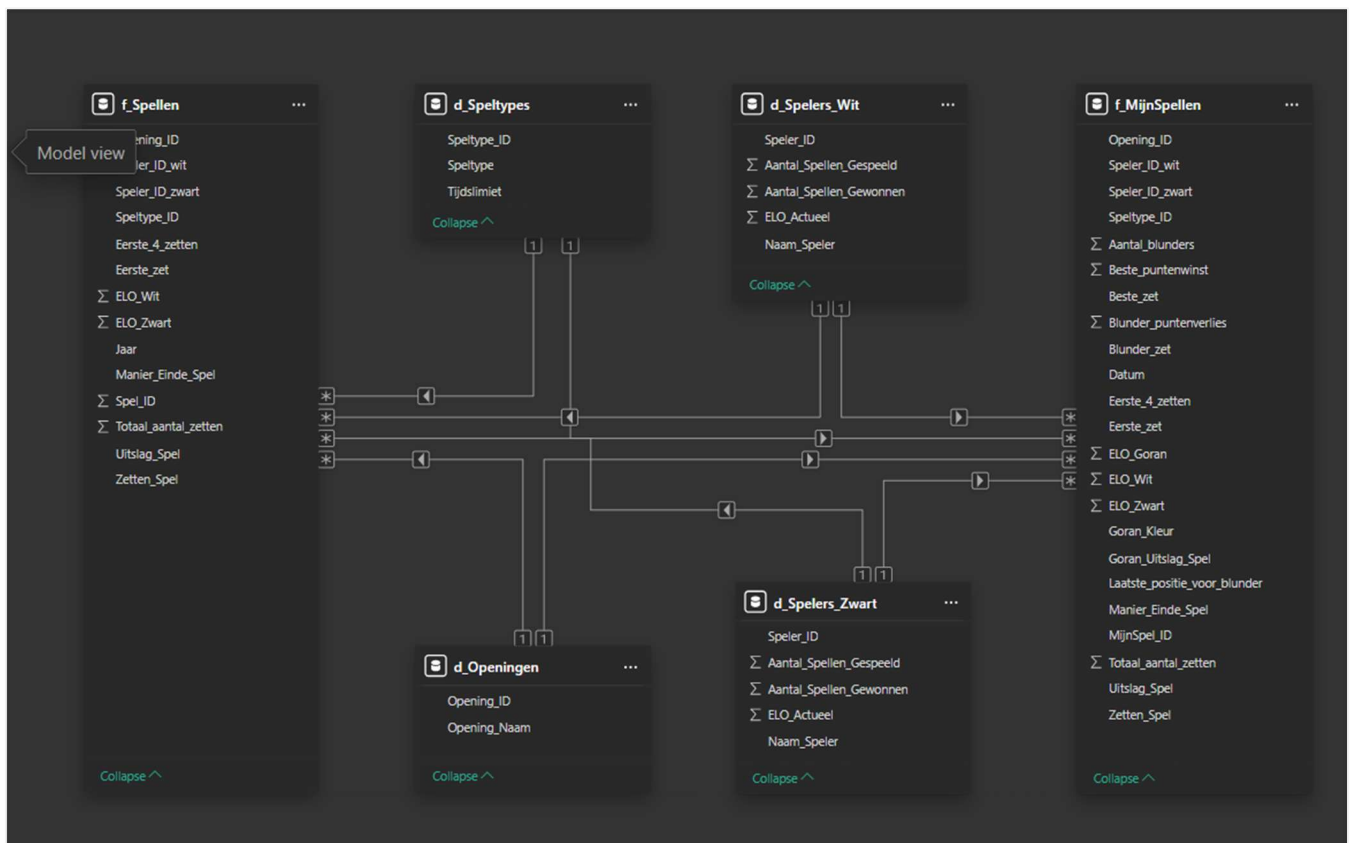
Bij het opmaken van de data mart stoot ik direct al op een probleem. Mijn feitentabellen 'Spellen' en 'MijnSpellen' hebben elk twee kolommen met speler ID's. Elke rij (= elk spel) heeft twee spelers die eender welke kleur (zwart of wit) kunnen zijn. Ik kan geen twee actieve relaties tegelijk leggen tussen deze twee tabellen en de 'Spelers' tabel.

Dit heb ik opgelost door de 'Spelers'-tabel te kopiëren. Dan heb ik de ene spelerstabel gelinkt aan beide feitentabellen hun zwarte speler ID. De andere is dan gelinkt aan beide feitentabellen hun witte speler ID.

De ene spelerstabel noem ik 'Spelers_Wit'. De andere 'Spelers_Zwart'.

Al de tabellen heb ik ook een voorzetsel gegeven. Feitentabellen beginnen met 'f_' en dimensietabellen beginnen met 'd_'.

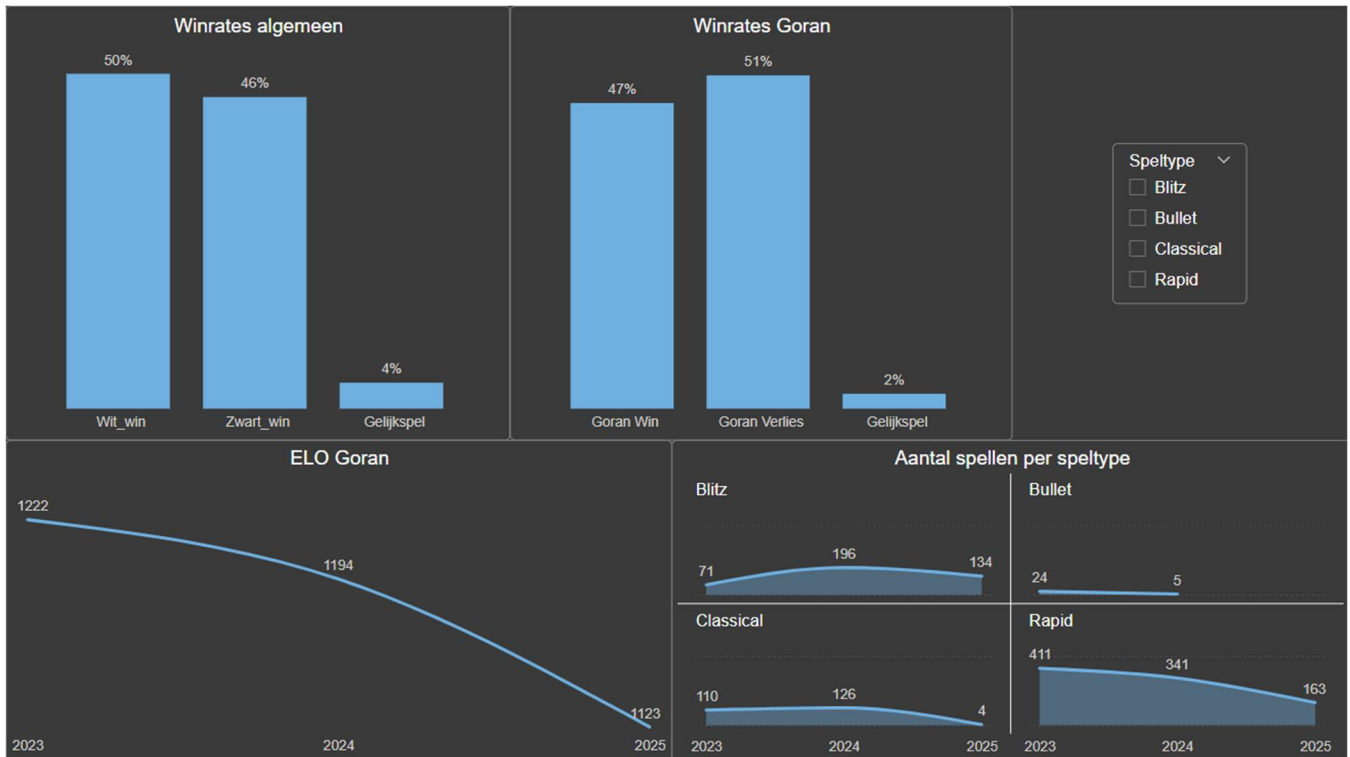
Normaal gezien worden de feitentabellen in het midden gezet en de dimensietabellen er rond. In deze situatie zag dat er te complex uit en daarom heb ik het omgekeerd gedaan, zoals je hieronder kan zien.



3.2. Mijn schaakspel verbetert niet. Integendeel!

Dit is de eerste pagina in Power BI. Hier wordt de huidige situatie / het probleem geschetst.

Wat kunnen we leren uit deze visuals, wetende dat alle spelers 50% kans hebben om wit of zwart te zijn (m.a.w., elke speler heeft ongeveer evenveel spellen met wit als met zwart):



De visuals die we hier zien waren enkel sleepwerk. Daarom lijkt het mij niet nodig om uit te leggen hoe ze gemaakt zijn (de paper is al zo uitgebreid).

Dat de gemiddelde speler een winpercentage van bijna 50% haalt voor alle speltypes in de laatste 3 jaar

In diezelfde periode (ik startte met schaken in 2023) heb ik een gemiddelde winrate van ongeveer 47% voor alle speltypes

Daarnaast is mijn aandeel gelijkspellen kleiner dan dat van de gemiddelde speler, waardoor ik ook extra vaak verlies.

Ik speel bijna nooit het speltype 'Classical', waar elke speler 30 min speeltijd heeft. De andere types zijn veel korter. Ik geef mezelf m.a.w. geen tijd om na te denken over mijn zetten en om fouten te onthouden.

Daarom dat mijn ranking (ELO) niet enkel stagneert, maar zelfs daalt doorheen de tijd. Mijn gemiddelde ELO was in 2023 nog 1222, nu is dat 1123. Ik heb 2 jaar meer schaakervaring dan toen ik begon en toch ben ik 100 ELO-punten gezakt sindsdien.

Hoe kan dat?

Vandaag de dag heb ik het gevoel dat er niet genoeg uren zijn in een dag.

In 2024 ben vader geworden, volg ik avondles, heb ik meer verantwoordelijkheden gekregen op het werk, ben ik terug beginnen fitnessen, en hebben wij ons huis gekocht, samen met een hond en een kat.

Ik kan zeker zeggen dat mijn leven er nu helemaal anders uitziet dan in 2023. Schaken staat onderaan mijn ladder van prioriteiten en gebeurt enkel wanneer ik 'een momentje' heb.

Omdat het leven drukker geworden is, ben ik tegenwoordig genoodzaakt om de kortdurende speltypes op te zoeken, in mijn geval 'Blitz' (3min per speler) en 'Rapid' (10min per speler). De Langdurige speltypes zoals 'Classical' (30min per speler) laat ik links liggen omdat ik die meestal halfweg moet stopzetten. Dat is duidelijk te zien in de visual 'aantal spellen per speltype'. Daarin staat ook het type 'Bullet' (1min per speler), maar dat vond ik altijd al te kort duren en heb ik bijna niet gespeeld in de laatste 2 jaar.

Als je opzoekt in Google hoe je beter kan worden in schaken, kom je vaak het advies tegen dat je de langdurige speltypes juist moet opzoeken (Silman, 2016). Zo leer je anticiperen, tactieken ontwikkelen en simpelweg zorgvuldig nadenken over je zetten. Dit alleen al voorkomt de meeste fouten en je herinnert je meer over deze partijen. De kortere spelvarianten vliegen voorbij en worden amper opgeslagen in ons geheugen, waardoor we er eigenlijk niet veel van bijleren.

Daarnaast wordt er sterk de nadruk gelegd op het analyseren van je schaakpartijen (Lu, 2021), (Shah, 2016). De analyse-engine 'Stockfish' bekijkt al je zetten en haalt de beste en de slechtste (blunders) eruit. Zo zie je waar het fout ging en wat de oplossing had geweest. Op die manier voorkom je dat je dezelfde fouten blijft maken.

Er moet ook plaats gemaakt worden voor het introduceren van theorie in je schaakpartijen. Dan hebben we het voornamelijk over het instuderen van schaakopeningen en het oplossen van schaakpuzzels (schaakmatpatronen, eindposities, vaak voorkomende tactieken, etc.).

Tenslotte moet je zeker ook van andere mensen leren schaken (Studer, 2021). Lees boeken over schaken en volg / bekijk partijen van grootmeesters.

Ik heb in die drie jaren schaken geen enkel van de bovengenoemde zaken gedaan. Dit in combinatie met mijn drukker leven is volgens mij de oorzaak van mijn 'ELO-regressie'.

3.3. Hoe word ik een betere schaakspeler?

3.3.1. Betrek theorie in je schaakspel

Op de tweede en derde pagina in Power BI probeer ik te achterhalen welke openingen er best toegevoegd worden aan mijn repertoire voor zowel zwart als wit.

3.3.1.1. Openingen voor zwart

We beginnen met de openingen voor zwart. Die wil ik ophalen a.d.h.v. de drie meest voorkomende eerste zetten uit alle schaakspellen in de tabel 'Spellen'.

Dit zijn m.a.w. de 3 meest voorkomende eerste zetten van wit waarop je moet reageren als zwart. Zwart heeft namelijk weinig tot geen beslissingsrecht als het aankomt op het kiezen van een opening. Geen enkele opening van zwart kan reageren op alle eerste zetten.

De 'King's Indian Defense' komt wel dicht in de buurt, maar die is redelijk ingewikkeld en is niet helemaal universeel toepasbaar. Deze opening heb ik ooit eens globaal bekeken en ik speel sindsdien altijd een geïmproviseerde versie hiervan. Ik gebruik die al zo lang omdat dit de meest universeel toepasbare opening was die ik destijds kon vinden en dat vond ik gemakkelijk. De 'King's Indian Attack' is trouwens heel gelijkaardig en ik gebruik die voor wit; opnieuw gebruik ik een slordige en geïmproviseerde versie van deze opening. Helaas heeft deze aanpak mij tot nu toe weinig opgeleverd. Ik denk niet meer na over mijn zetten en speel steeds dezelfde formatie, waardoor ik kansen en risico's uit het oog verlies.

Ik wil de drie meest voorkomende eerste zetten tonen, met daarbij het aantal keren dat die voorkomen, uitgedrukt in '% van het grote totaal'.

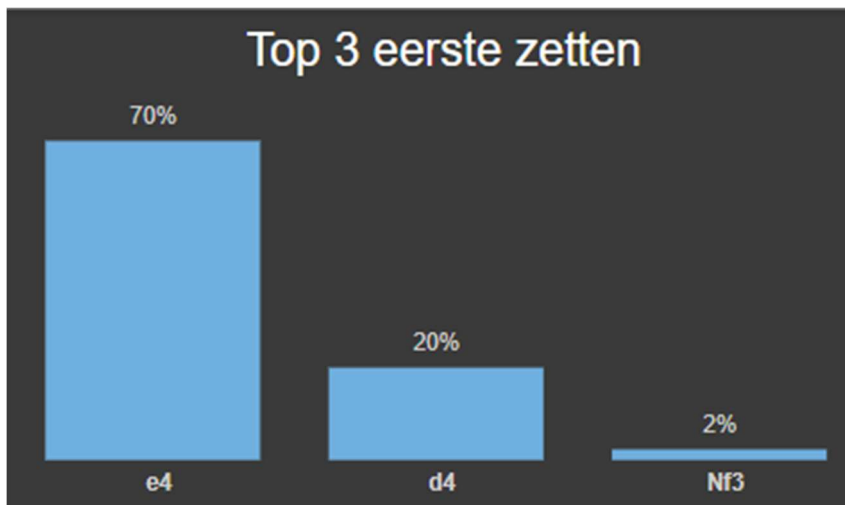
Met een 'Top N' filter kreeg ik het % van het totaal van enkel die drie zetten te zien, maar dat geeft een vertekend beeld. Dit percentage moet in verhouding zijn tot het totale aantal eerste zetten in de volledige tabel 'Spellen'.

Dit heb ik kunnen oplossen met enkele measures:

- AantalSpellen = COUNT(f_Spellen[Spel_ID])
- Topdriezetten_aandeeltotaal = DIVIDE([AantalSpellen]; CALCULATE([AantalSpellen]; ALL(f_Spellen[Eerste_zet])))
- Topdriezetten_rang = RANKX(ALL(f_Spellen[Eerste_zet]); [AantalSpellen]; ;DESC)

De kolom 'Spellen_Eerste_zet' en de measure 'Topdriezetten_aandeeltotaal' zijn toegevoegd aan de visual. De measure 'Topdriezetten_rang' is ingesteld als filter op niveau van de visual, die ingesteld staat op 'is less than or equal to' → '3'.

Nu zien we de 3 meest voorkomende eerste zetten en daarbij het juiste % van het grote totaal.



Om de 3 beste openingen van zwart tegen deze 3 zetten te vinden heb ik terug wat extra's moeten doen.

Enkele measures toevoegen aan de 'Spellen' tabel:

TotaalSpellen = COUNTROWS(f_Spellen)

➤ TotaalSpellen_per_Opening =

```
CALCULATE(  
    COUNTROWS(f_Spellen);  
    ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet]; d_Openingen[Opening_naam])  
)
```

➤ Zwartwin_per_Opening =

```
CALCULATE(  
    COUNTROWS(f_Spellen);  
    f_Spellen[Uitslagspel] = "Zwart_win";  
    ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet]; d_Openingen[Opening_naam])  
)
```

```

➤ Zwart_Winrate =
DIVIDE(
    [Zwartwin_per_Opening];
    [TotaalSpellen_per_Opening]
)
➤ Opening_Percentage =
VAR TotaalVoorZet =
    CALCULATE(
        COUNTROWS(f_Spellen);
        ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet])
    )
RETURN
    DIVIDE([TotaalSpellen_per_Opening]; TotaalVoorZet)

```

De beste openingen voor zwart wil ik visualiseren met behulp van 3 aparte visuals. Op die manier kunnen we de 3 eerste zetten duidelijk onderscheiden van elkaar.

De visuals zijn matrices:

- Rijen: namen van de openingen
- Waarden: de winrate van zwart & de speelfrequentie van de openingen tegen die bepaalde eerste zet
- Er wordt gefilterd op die bepaalde eerste zet, met een top 3 filter op de winrate van zwart en tenslotte nog een filter op de speelfrequentie van de opening. Die opening moet voor mij tevens in minstens 5% van de gevallen gespeeld worden (opening die weinig gespeeld worden kunnen een heel hoge winrate hebben en die moeten eruit).

Hier gaat nog iets fout. Het combineren van een top N filter met andere filters geeft geen enkel resultaat. Power BI geeft voorrang aan de verkeerde filters.

Opgelost door nog een measure toe te voegen:

```

➤ Zwart_Winrate_Gefilterd =
IF(
    [Opening_Percentage] >= 0,05;

```

[Zwart_Winrate];

BLANK())

Deze measure zet ik in als waarde bij de filter 'Opening_naam' op niveau van de visual met de selectie Top N = 3. De filter op de speelfrequentie (meer dan 5%) heb ik gewist, want die zit al vervat in deze laatste measure.

Opening tegen e4		
Opening	Winrate zwart	Frequentie
French Defense, Irregular Responses	48%	7%
Sicilian defense	46%	5%
King's Pawn Game	45%	7%

Opening tegen d4		
Opening	Winrate zwart	Frequentie
Queen's Pawn Game, Trompowsky Attack	49%	6%
Queen's Pawn Game, 2. Nf3	46%	7%
Closed Game, Irregular Responses	45%	15%

Opening tegen Nf3		
Opening	Winrate zwart	Frequentie
Queen's Pawn Game, 2. Nf3	49%	10%
Zukertort Opening	47%	41%
Réti Opening	46%	9%

Dit geeft al een goed beeld van wat ik zoek, maar misschien is het handiger om dieper te reiken. Ik wil dit bekijken per ELO-bereik van 200 punten. Voornamelijk wil ik natuurlijk de resultaten zien die binnen mijn 'ELO range' vallen ('1001-1200' en/of '1201-1400').

Daarom heb ik aan de tabel 'Spellen' een berekende kolom toegevoegd:

➤ ELO_Range =

VAR gem_ELO = DIVIDE(f_Spellen[ELO_Wit] + f_Spellen[ELO_Zwart]; 2)

VAR Ondergrens = INT((gem_ELO - 1) / 200) * 200 + 1

VAR Bovengrens = Ondergrens + 199

RETURN

Ondergrens & "-" & Bovengrens

De ELO-ranges zijn alfabetisch gesorteerd en dat zet de bereiken dus door elkaar. Dit kan opgelost worden adhv een hulpkolom:

➤ ELO_Sorteren =

INT((DIVIDE(f_Spellen[ELO_Wit] + f_Spellen[ELO_Zwart]; 2) - 1) / 200)

Nu kan ik in de data view naar de tabel 'Spellen' gaan en de kolom 'ELO_Range' sorteren o.b.v. kolom 'ELO_Sorteren'.

De ELO-ranges staan nu mooi gesorteerd en ik voeg die toe als slicer op deze pagina. Hier selecteer ik de twee ELO-bereiken '1001-1200' en '1201-1400'.

De data in mijn tabel 'spellen' is verdeeld over 10 jaren. Ik zou graag meer actuele data zien. Ik voeg nog een slicer in op de pagina en maak een selectie van de laatste 2 jaren.

Tenslotte voeg ik nog een laatste slicer in op de pagina, namelijk 'Speltype'. Ik wil, zoals aangeraden wordt, stoppen met de kortdurende speltypes 'Bullet' en 'Blitz' (ik speel eigenlijk sowieso al bijna geen Bullet).

In de laatste pagina van Power BI (analyse op mijn eigen spellen) bekijken we de speltypes nog even, vooral om na te kijken of ik effectief minder fouten bega als er langere tijdslimieten zijn.

Samengevat: we zien hier de beste openingen voor zwart om te reageren op de drie meest voorkomende eerste zetten, binnen mijn ELO-bereik, tijdens de laatste twee jaren, voor de langdurige speltypes.

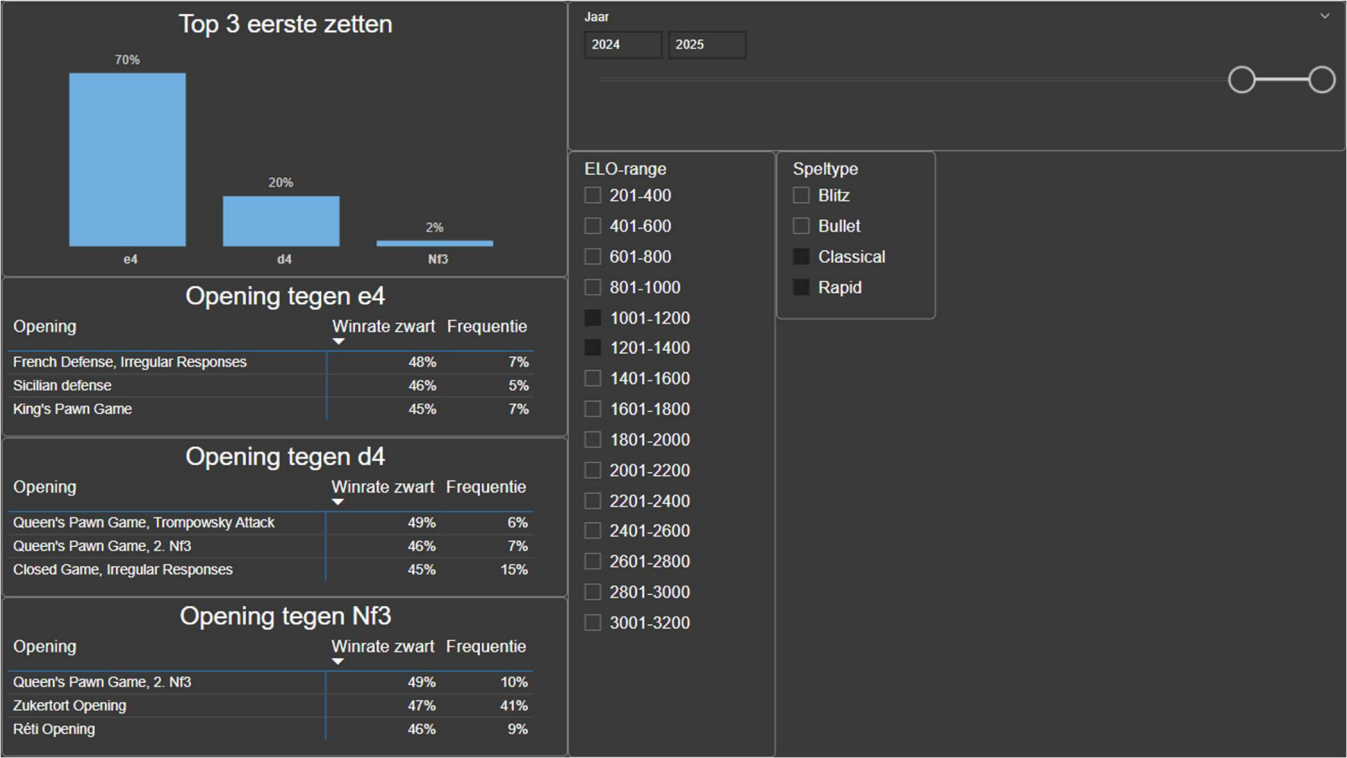
De meest voorkomende eerste zetten , met daarbij de beste openingen voor zwart zijn als volgt:

- **E4 → French Defense**
- **D4 → Queen's Pawn game**
- **Nf3 → Queen's Pawn game (of de Zukertort opening?)**

Ik moet in principe maar twee openingen leren gebruiken als zwarte speler om mezelf te wapenen tegen de drie meest voorkomende zetten (*wel interessant: de op een na hoogste winrate tegen Nf3 is de Zukertort opening. Die wordt tegen deze zet 41% van de keren gespeeld*).

Wanneer er een andere eerste zet wordt gespeeld, zal ik nog steeds moeten improviseren (wat ik tot nu toe altijd al deed). Deze aanpak is niet waterdicht, maar het zal mij zeker op weg helpen om een betere schaakspeler te worden.

Nu gaan we op de volgende pagina over naar de beste openingen voor mij als witte speler.



Speltype

☐ Blitz

☐ Bullet

☒ Classical

☒ Rapid

3.3.1.2. Openingen voor wit

Als witte speler is het een stuk makkelijker om de beste openingen te vinden. Omdat je als eerste aan zet bent, kies je steeds zelf welke opening er gespeeld wordt.

Die drie openingen wil ik op een gelijkaardige manier vinden. Opnieuw zoeken we naar drie openingen met de hoogste winrate voor wit die minstens **1%** van de keren gespeeld worden (deze ondergrens heb ik verlaagd van 5% naar 1% omdat we nu een véél grotere selectie van openingen hebben. 1% speelfrequentie op +-2 miljoen schaakspellen is al 20.000 spellen).

Daarnaast wil ik graag een gevarieerd openingenrepertoire hebben, dus elk van die drie openingen moet een verschillende eerste zet hebben.

Eerst en vooral wil ik zien welke eerste zetten er de hoogste winrate geven voor wit. Hier ook mits een minimum van 1% speelfrequentie. De measures zijn als volgt:

➤ Wit_Winrate_Per_EersteZet =

VAR WitWins =

```
CALCULATE(  
    COUNTROWS(f_Spellen);  
    f_Spellen[Uitslagspel] = "Wit_Win"  
)
```

VAR TotalGames =

```
COUNTROWS(f_Spellen)
```

RETURN

```
DIVIDE(WitWins; TotalGames)
```

➤ EersteZet_Percentage =

VAR AantalSpellenVoorZet =

```
CALCULATE(  
    COUNTROWS(f_Spellen);  
    ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet])  
)
```

VAR TotaalSpellen =

```
CALCULATE(  

```

```

COUNTROWS(f_Spellen);
ALL(f_Spellen)
)
RETURN
DIVIDE(AantalSpellenVoorZet; TotaalSpellen)

```

➤ Wit_Winrate_Gefilterd =

```

IF(
[EersteZet_Percentage] >= 0,01;
[Wit_Winrate_Per_EersteZet];
BLANK()
)

```

➤ Rang_EersteZet_Winrate =

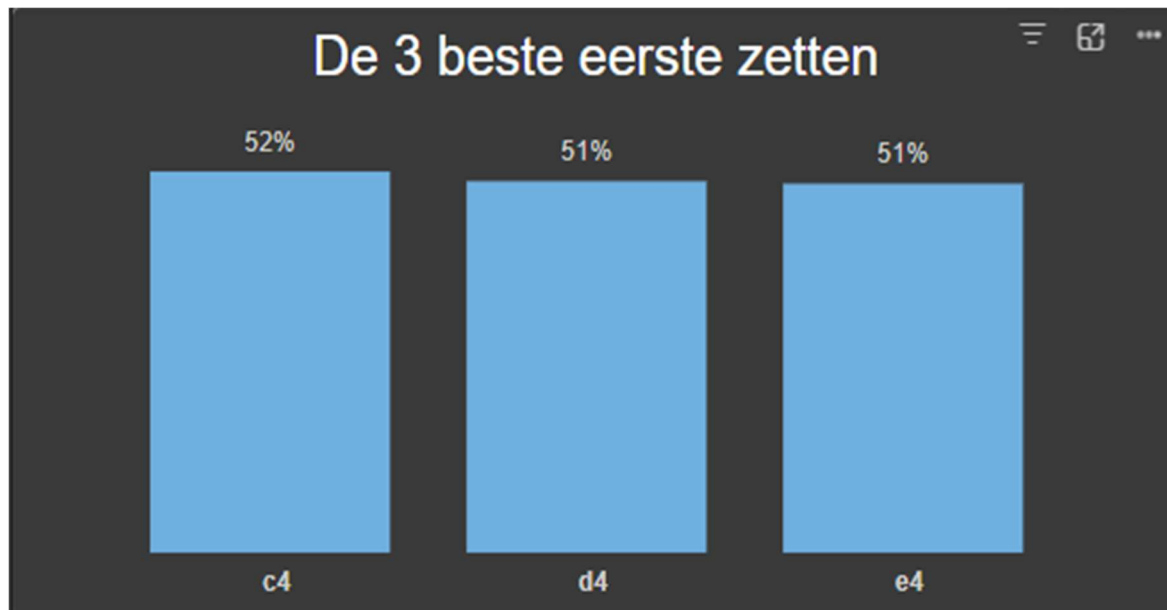
```

RANKX(
ALLSELECTED(f_Spellen[Eerste_zet]);
[Wit_Winrate_Gefilterd];
;
DESC;
DENSE
)

```

Om dit te visualiseren heb ik een 'column chart' gebruikt

- X-as = Eerste_zet
- Y-as = Wit_Winrate_gefilterd
- Filter op deze visual = Rang_EersteZet_Winrate (is kleiner of gelijk aan 3)



Voor elk van deze 3 zetten wil ik de openingen met de hoogste winrate voor wit vinden.

Hier kan bijna dezelfde werkwijze gebruikt worden als voor de zwarte openingen.

De measures:

➤ TotaalSpellen_per_Opening_Wit =

```
CALCULATE(
    COUNTROWS(f_Spellen);
    ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet]; d_Openingen[Opening_naam])
)
```

➤ Witwin_per_Opening =

```
CALCULATE(
    COUNTROWS(f_Spellen);
    f_Spellen[Uitslagspel] = "Wit_Win";
    ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet]; d_Openingen[Opening_naam])
)
```

➤ Wit_Winrate2 =

```
DIVIDE(
```

```

[Witwin_per_Opening];
[TotaalSpellen_per_Opening_Wit]
)

➤ Opening_Percentage_Wit =
VAR TotaalVoorZet =
    CALCULATE(
        COUNTROWS(f_Spellen);
        ALLEXCEPT(f_Spellen; f_Spellen[Eerste_zet])
    )
RETURN
    DIVIDE([TotaalSpellen_per_Opening_Wit]; TotaalVoorZet)

➤ Wit_Winrate_Gefilterd2 =
IF(
    [Opening_Percentage_Wit] >= 0.05;
    [Wit_Winrate];
    BLANK()
)

```

De visuals zijn matrices:

- Rijen = naam van de opening
- Waarden = de winrate van wit + de speelfrequentie voor die openingen
- Filters = Telkens één van de drie beste eerste zetten & de naam van de opening gefilterd op de top 3 met als waarde Wit_Winrate_Gefilterd2

Opening met c4		
Opening	Winrate wit	Frequentie
English Opening, Symmetrical	62%	7%
English Opening, Symmetrical variation	57%	4%
English Opening, Four knights system	54%	1%

Opening met d4		
Opening	Winrate wit	Frequentie
QGD, 3.Nc3	59%	2%
Dutch defense	58%	2%
QGD 3...Nf6	56%	1%

Opening met e4		
Opening	Winrate wit	Frequentie
King's Gambit	56%	1%
King's Knight Opening	56%	3%
Alekhine's defense	55%	2%

De slicers zijn dezelfde als die bij de zwarte openingen.

Jaar

2024

2025

ELO-range
☐ 201-400
☐ 401-600
☐ 601-800
☐ 801-1000
☒ 1001-1200
☒ 1201-1400
☐ 1401-1600
☐ 1601-1800
☐ 1801-2000
☐ 2001-2200
☐ 2201-2400
☐ 2401-2600
☐ 2601-2800
☐ 2801-3000
☐ 3001-3200

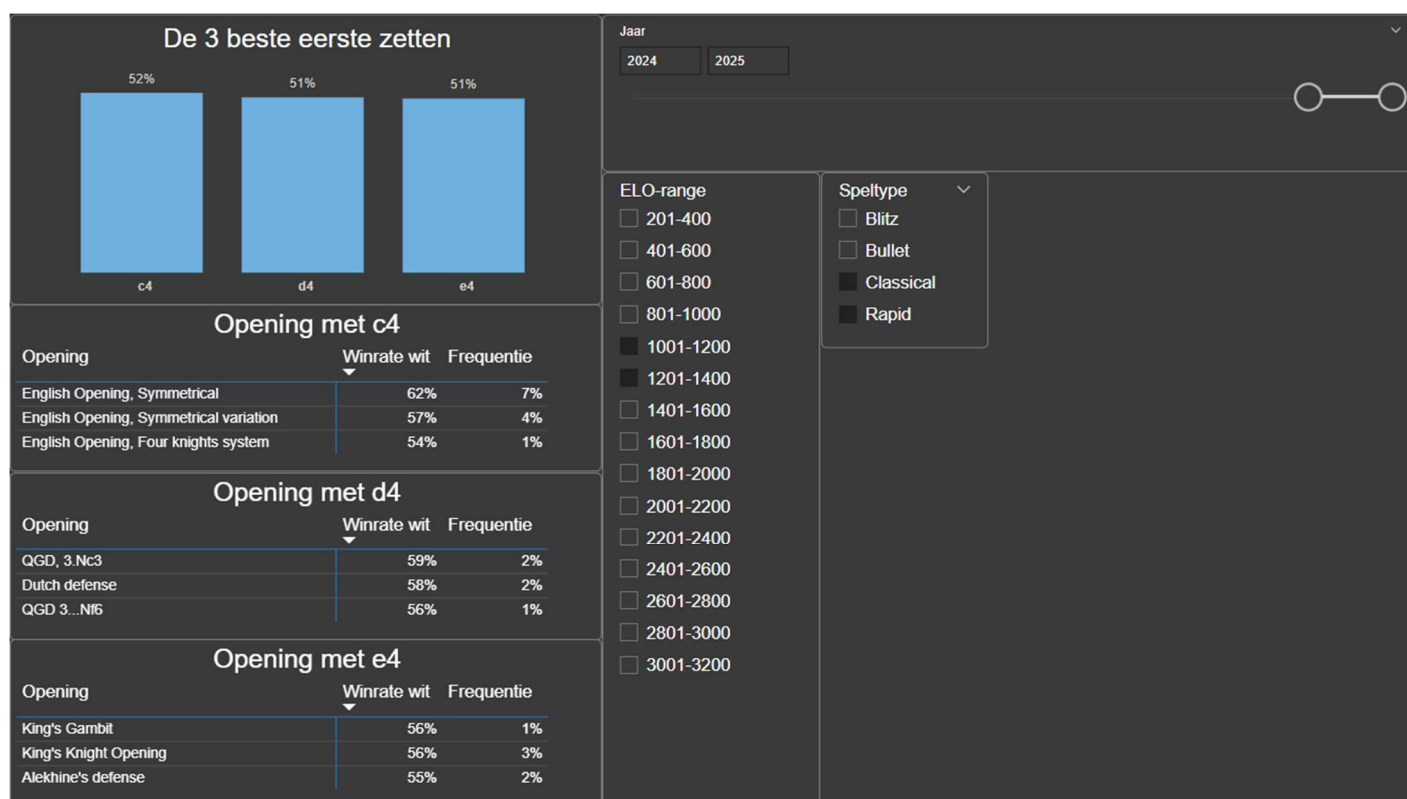
Speltype
☐ Blitz
☐ Bullet
☒ Classical
☒ Rapid

Samengevat: we zien hier de beste openingen voor wit op basis van de drie meest succesvolle eerste zetten voor wit. Dit binnen mijn ELO-bereik, tijdens de laatste twee jaren, voor de langdurige speltypes.

De meest succesvolle eerste zetten voor wit, met daarbij de beste openingen voor wit zijn als volgt:

- **C4 → English Opening**
- **D4 → Queen's Gambit (Declined)**
- **E4 → King's Gambit**

Nu ik voor zowel zwart als wit de drie beste openingen gevonden heb. Is het tijd om mijn eigen schaakspellen te analyseren, met als einddoel de beste schaakpuzzelcategorieën te vinden waarop ik kan oefenen.



3.3.2. Analyseer je eigen spellen

Ik heb een analyse uitgevoerd op mijn schaakspellen met de analyse-engine 'Stockfish'. Daaruit haalde ik telkens onder andere mijn slechtste zet ('blunder') uit en ook de positie op het schaakbord net voor die blunder.

Wat wil ik bereiken met die informatie: ik wil de posities in het schaakspel kunnen achterhalen waar de blunder juist gebeurd is. Daarna wil ik de blunders in deze posities een categorie kunnen geven (vb. Slechte ruil / gemiste schaakmat / fork / tempoverlies enz.). Tenslotte wil ik de drie meest voorkomende categorieën tonen in een visual.

Waarom deze analyse: Deze drie categorieën wil ik oefenen in schaakpuzzels om zo mijn grootste zwaktes in het schaakspel aan te pakken.

Om deze analyse te kunnen doen moet er eerst nog een python script uitgevoerd worden en, gezien de categorisering van deze blunders nogal een menselijke interpretatie is, in combinatie met een prompt aan OpenAI.

Ik heb een csv-file nodig met daarin de primary key van de tabel 'Mijnspellen', met daarnaast mijn slechtste zet per spel en de positie net voor die blunder. Die heb ik geëxporteerd uit Power BI en opgeslagen in pad "C:\Eindwerk\mijn_blunders.csv".

Dan het python script opgeslaan in pad "C:\Eindwerk\blunder_analyse.py"

Hier is de python-code:

```
➤ import openai
```

```
import pandas as pd
```

```
import time
```

```
# HIER STAAT DIE API-KEY
```

```
client = openai.OpenAI(api_key="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")
```

```
INPUT_CSV = r"C:\Eindwerk\mijn_blunders.csv"
```

```
OUTPUT_CSV = r"C:\Eindwerk\mijn_blunders_met_categorie.csv"
```

```
BLUNDER_CATEGORIEEN = [
```

```
    "fork",
```



```
"pin",
"skewer",
"discovered attack",
"stuk zonder dekking",
"slechte ruil",
"gemiste mat",
"batterij genegeerd",
"tempoverlies",
"slechte promotie",
"afleiding",
"dubbele aanval",
"verdediging genegeerd"
]
```

Prompt aan openAI

```
def categorie_bepalen(fen, zet):
```

```
    prompt = f"""
```

Je bent een schaaktrainer. Analyseer de fout in onderstaande stelling en zet:

FEN: {fen}

Zet: {zet}

Kies één van deze fouttypes (exact zo noteren):

```
{', '.join(BLUNDER_CATEGORIEEN)}
```

Als geen enkele past, antwoord: "onbekend"

Geef alleen het fouttype terug. Geen uitleg.

"""

```
try:
    response = client.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}],
        temperature=0,
        max_tokens=20
    )
    categorie = response.choices[0].message.content.strip().lower()
    if categorie not in [c.lower() for c in BLUNDER_CATEGORIEEN] + ["onbekend"]:
        return "onbekend"
    return categorie
except Exception as e:
    print(f"Fout bij API-aanroep: {e}")
    return "onbekend"
```

Data inladen in de csv-file

```
df = pd.read_csv(INPUT_CSV)
```

```
df["Blunder_categorie"] = None
```

Over de blunders gaan

```
for idx, row in df.iterrows():
```

```
    fen = row.get("Laatste_positie_voor_blunder")
```

```
    zet_raw = row.get("Blunder_zet")
```

```
if pd.isna(fen) or pd.isna(zet_raw):
```

```
    df.at[idx, "Blunder_categorie"] = "onbekend"
```

```
    continue
```

```
zet = zet_raw.split("-")[1] if "-" in zet_raw else zet_raw
```

```
fouttype = categorie_bepalen(fen, zet)
```

```
df.at[idx, "Blunder_categorie"] = fouttype
```

```
print(f"Blunder {idx+1}/{len(df)} → {zet} → {fouttype}")
```

```
time.sleep(1.1)
```

```
# csv-file opslaan
```

```
df.to_csv(OUTPUT_CSV, index=False)
```

```
print(f"\n Analyse voltooid! Resultaat: {OUTPUT_CSV}")
```

Nu het python-script uitvoeren in powershell:

- pip install openai *(ik had 1 pakket nog niet)*
- cd C:\eindwerk
- python blunder_analyse.py

De blundercategorieën zijn nu toegevoegd en opgeslagen als een csv-file in pad

"C:\Eindwerk\mijn_blunders_met_categorie.csv".

Dit brengen we terug naar Power BI via Power Query (csv inladen → left outer merge met 'f_MijnSpellen' o.b.v. 'Mijnspel_ID').

Tenslotte nog een measure voor de visual:

- Blunder_Percentage_Van_Totaal =

VAR TotaalBlunders =

CALCULATE(

COUNTROWS(f_MijnSpellen);

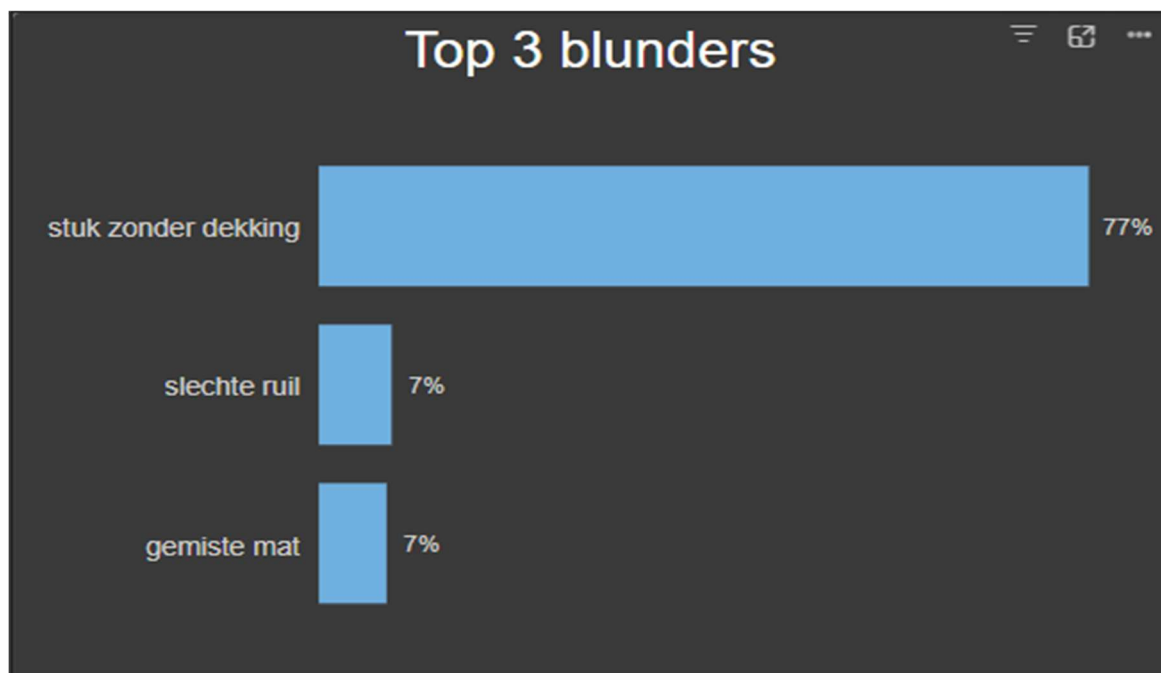
```

REMOVEFILTERS(f_MijnSpellen[Blunder_categorie]))
RETURN
DIVIDE(
    COUNTROWS(f_MijnSpellen);
    TotaalBlunders
)

```

De visual is een bar chart:

- Y-as = de blunder categorie
- X-as = het % van voorkomen van die blunder t.o.v. het totale aantal spellen
- Filters: de blunder categorie 'onbekend' eruit gehaald & de top 3 blunder categorieën geselecteerd o.b.v. het aantal keren dat de blunder categorie voorkomt



Deze drie **blundercategorieën** zijn een belangrijke selectie voor de schaakpuzzels die ik ga oefenen.

Wat ik mij daarnaast nog afvraag, is in welke fase van het schaakspel mijn grootste zwaktes liggen. Met ‘fase’ bedoel ik opening / middenspel / eindspel.

Dit kunnen we ook achterhalen met de Stockfish-data. Ik moet voor elke rij het nummer van de blunderzet eruit halen en voor elke rij nog een spelfase toevoegen. Hiervoor voegen we drie berekende kolommen toe:

➤ BlunderZetNummer =

```
VAR NummerZet = 'f_Mijnspellen'[Blunder_zet]
```

```
RETURN
```

```
VALUE(LEFT(NummerZet; FIND("-", NummerZet) - 1))
```

➤ BlunderPositieRelatief =

```
DIVIDE('f_Mijnspellen'[BlunderZetNummer]; 'f_Mijnspellen'[Totaal_aantal_zetten])
```

➤ PartijfaseBlunder =

```
SWITCH(
```

```
TRUE();
```

```
ISBLANK('f_Mijnspellen'[BlunderPositieRelatief]); "Onbekend";
```

```
'f_Mijnspellen'[BlunderPositieRelatief] <= 0,33; "Opening";
```

```
'f_Mijnspellen'[BlunderPositieRelatief] <= 0,66; "Middenspel";
```

```
"Eindspel")
```

Als laatste nog een measure voor het aantal rijen in de ‘Mijnspellen’ tabel en eentje om het gemiddelde puntenverlies van de blunders te berekenen:

➤ Aantalblunders = COUNT('f_MijnSpellen'[MijnSpe_ID])

➤ GemiddeldPuntenverlies = AVERAGE('f_Mijnspellen'[Blunder_puntenverlies])

Omdat het puntenverlies van de blunders in negatieve getallen uitgedrukt is, zal dit gedeelte van de visual misleidend zijn. Daarom nog een measure om het puntenverlies om te zetten naar absolute getallen:

GemiddeldPuntenverliesAbsoluut =

```
AVERAGEX(
```

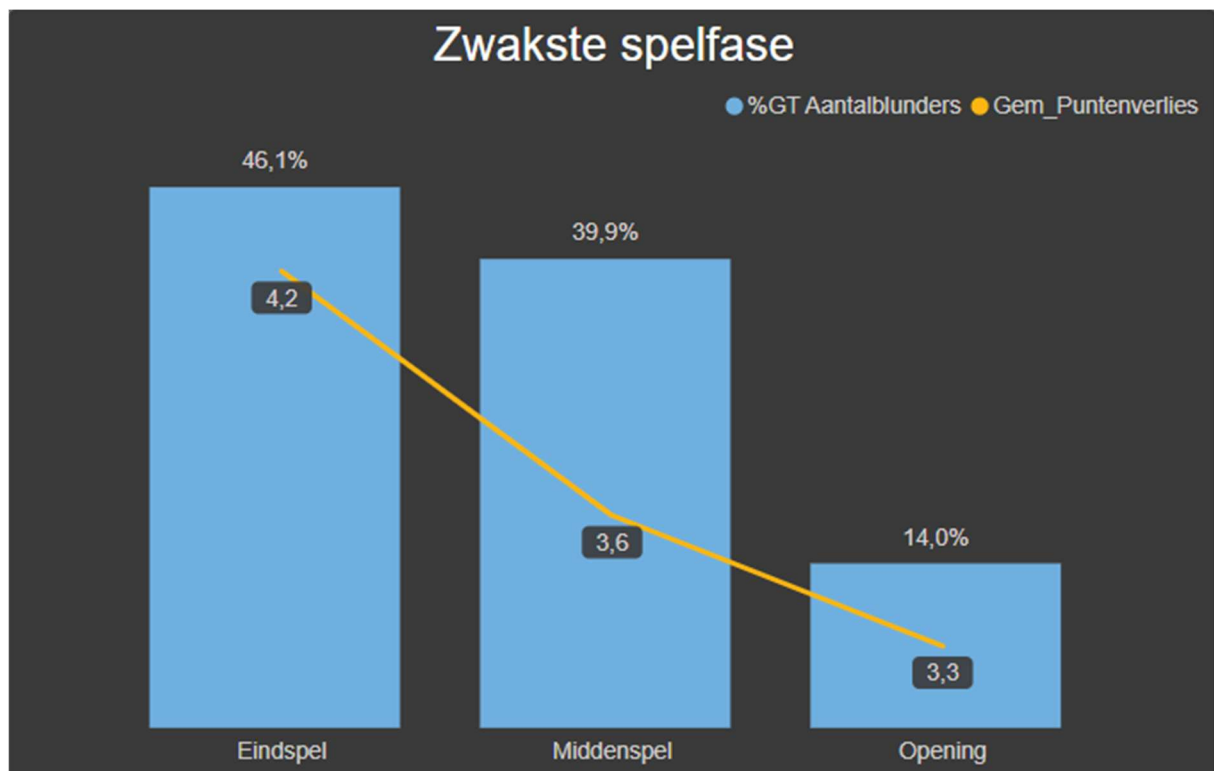
'f_Mijnspellen',

ABS('f_Mijnspellen'[Blunder_puntenverlies]))

De visual is een 'Line and clustered column chart':

(Kleine opmerking: de benamingen in de legende kunnen niet rechtstreeks aangepast worden en 'GemiddelpuntenverliesAbsoluut' is een lange naam. Ik heb daarom deze measure hernoemd naar 'Gem_Puntenverlies'.)

- X-as = In welke partijfase de blunder zich bevindt
- Kolom Y-as = Het aantal rijen van de 'mijnspellen' tabel (wordt 'Aantalblunders' genoemd, maar er wordt gewoon naar de ergste blunder per spel gekeken, dus in deze visual is elke rij één blunder)
- Lijn Y-as = Het absolute getal van het gemiddelde puntenverlies van de blunders
- Filters = Nvt



We kunnen hier duidelijk zien dat het grootste deel van slechtste zetten per spel in de eindfase gebeuren. Daarbovenop wegen de blunders die ik maak in de eindfase ook nog eens het zwaarst door (4,2 punten verliezen met één blunder is net iets erger dan in één zet een loper + pion kwijtraken).

Op de eerste pagina in Power BI zagen we ook dat ik een lagere winrate heb dan de gemiddelde schaakspeler en ik daarnaast ook minder vaak gelijkspel haal. Daarom heb ik een extra hoge verliesrate. Een verloren positie kunnen omzetten in gelijkspel is een belangrijke skill van een schaakspeler. Dit valt ook onder de eindfase.

Daarom moet ik zeker en vast eindspellen meenemen in mijn selectie van schaakpuzzels.

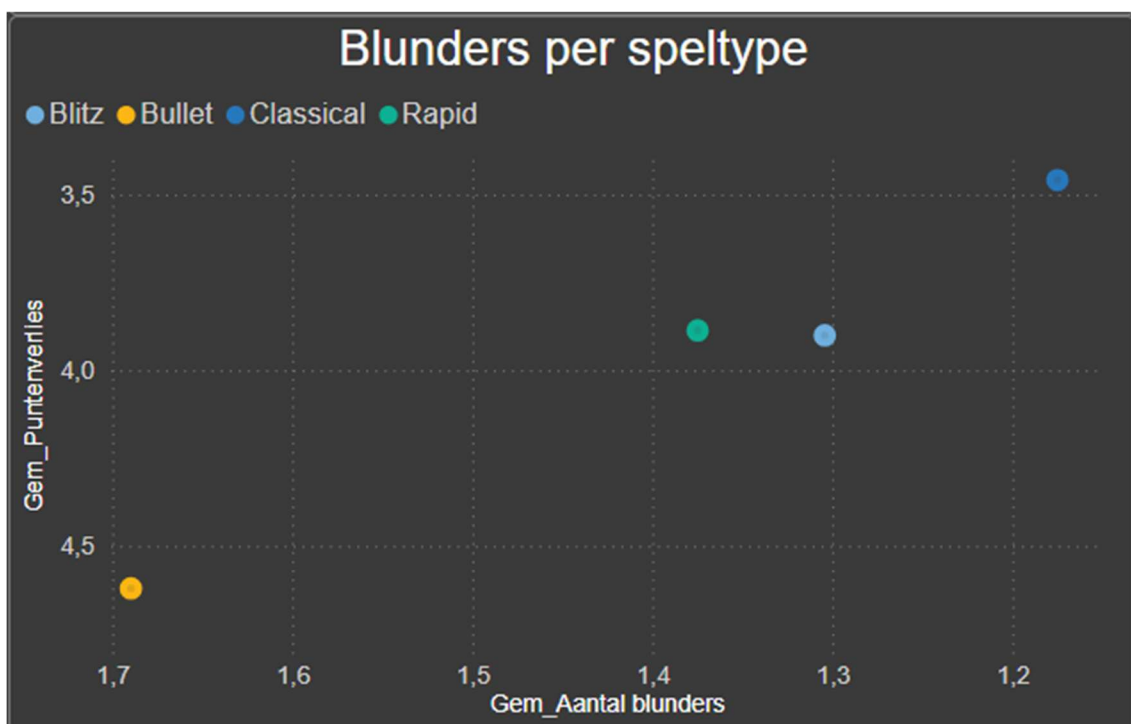
Als laatste wil ik de analyse afsluiten door nog een kijkje te nemen naar mijn blunders per speltype. Er wordt aangeraden om langere speltypes te spelen, onder andere omdat je dan minder fouten maakt / minder grote fouten maakt (Silman, 2016).

Ik vraag mij af of dat ook klopt voor mij. Dit is een louter een kleine controle op het gegeven advies. Het leuke aan een data analyse is dat je niet zomaar alles moet geloven wat er gezegd wordt. Je kan het 'fact-checken'.

Dit is een 'Scatter chart':

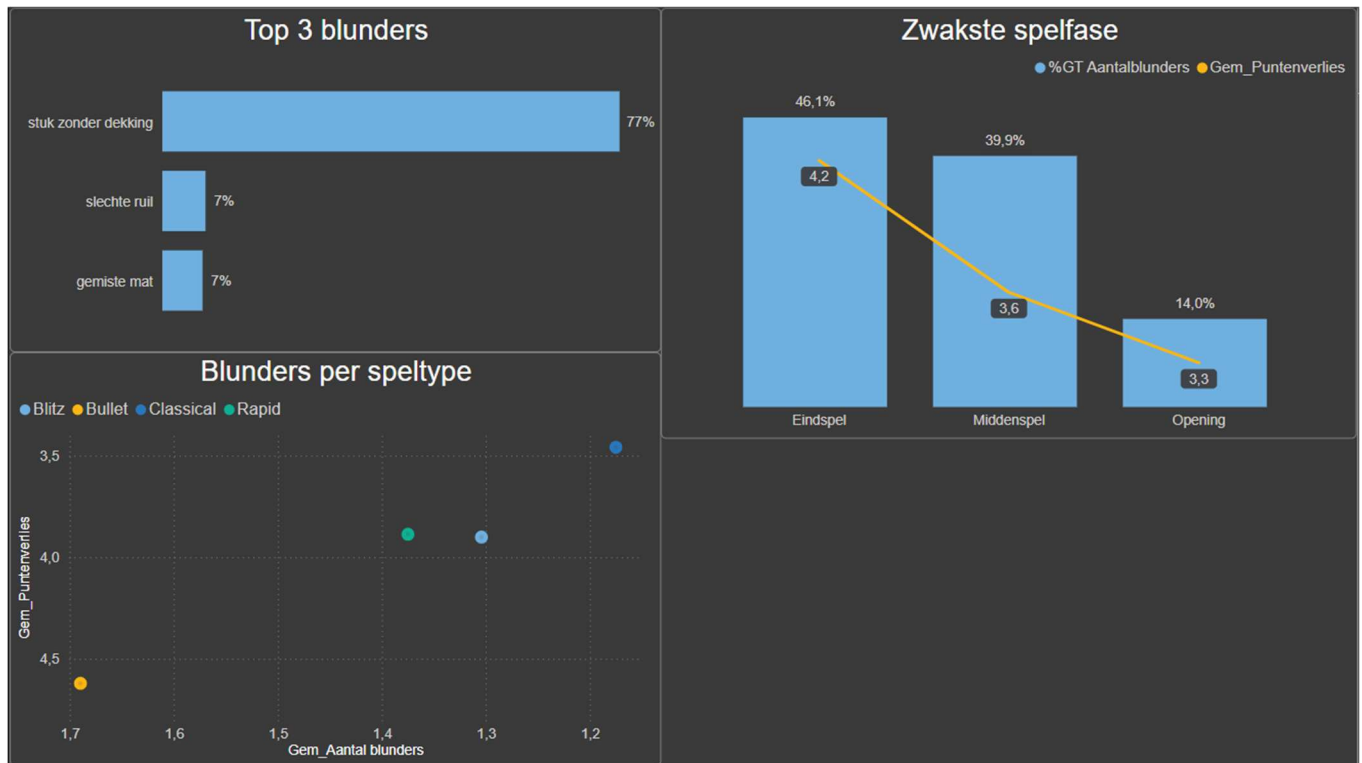
Ik heb de X- en Y-as omgedraaid, zodat het speltype waarin ik de minste fouten maak rechtsboven staat.

- X-as = het gemiddelde aantal blunders per speltype
- Y-as = Het gemiddelde puntenverlies van de grootste blunder van elk spel per speltype
- Legende = Speltype
- Filters = Nvt



Zoals verwacht zien we dat het (ongeveer) klopt dat ik minder fouten maak in de langere speltypes.

Blitz en rapid zijn redelijk gelijk, maar het is duidelijk dat ik het minst accuraat speel in het kortste speltype (Bullet) en het meest accuraat speel in het langste speltype (Classical).



Hier stopt het analysegedeelte van mijn onderzoek. Ik heb nu genoeg bouwstenen om mijn plan van aanpak samen te stellen.

4. Conclusies

De analyse is succesvol. Ik heb de antwoorden gevonden die ik zocht. Nu is het tijd voor actie!

4.1. Plan van aanpak

Dit onderzoek was niet hypothetisch. Het onderstaande actieplan ga ik effectief uitvoeren. Het plan is als volgt:

- Vanaf nu speel ik enkel nog de langere speltypes en bij voorkeur het type 'Classical'. De kortste types ('Bullet' en 'Blitz') raak ik niet meer aan.
- Elke partij analyseer ik na afloop met Stockfish. Deze engine is ingebouwd bij Lichess. Met één druk op de knop gaat het afgelopen spel over naar de analysemodus, waar ik meteen kan zien wanneer ik een blunder ben gegaan. De engine geeft telkens ook aan wat wél een goede zet (eigenlijk de 'beste' zet) had geweest.
- De zes beste openingen voor mij ga ik stuk voor stuk instuderen en dan enkel die posities spelen, tenzij ik niet anders kan (bij zwart zal dat soms het geval zijn).

De twee onderstaande acties zal ik voornamelijk uitvoeren voordat ik aan een spel begin of wanneer ik een momentje vrij heb (dat te kort is om een volledige partij te spelen).

- De selectie schaakpuzzels oefenen:
 - Stuk zonder dekking ('hanging piece')
 - Slechte ruil (bijvoorbeeld een toren opofferen om een paard te kunnen slaan)
 - Gemiste mat (Schaakmatpatronen)
 - De eindfase van het spel (mijn zwakste spelfase)(pionnen, promotieraces, schaakmatpatronen, gelijkspel forceren, enzovoort)
 - De zes beste openingen voor mij
- Een partij bekijken van een grootmeester die één of meer van mijn zes openingen meester is (en in die partij ook effectief die opening speelt)
 - Bvb. voor de 'Queen's Gambit Declined' zijn er enkele wereldtoppers die deze opening in hun standaardgamma hebben, zoals Vladimir Kramnik, Magnus Carlsen en Anatoly Karpov. Deze spelers hun partijen staan ook in grote aantallen op YouTube / het internet en soms zelfs met aanvullende commentaar van een andere schaakexpert (Jozarov, 2022).

Het uitvoeren van deze vijf actiepunten zal mijn schaakspel zeker verbeteren. Ik kijk er al met veel plezier naar uit!

5. Bibliografie

- GothamChess. (2020, 09 09). *Youtube*. Opgehaald van Youtube.com:
<https://www.youtube.com/watch?v=LWzltpXk2mg>
- Jozarov. (2022, 12 12). *Youtube*. Opgehaald van Youtube.com:
<https://www.youtube.com/watch?v=NhqWUhnxxrc>
- Kim, N. (2024, 09 13). *Blogs*. Opgehaald van Chess.com:
<https://www.chess.com/blog/Naoki71/the-secret-to-chess-improvement-in-just-6-steps?>
- Lichess. (sd). *Lichess*. Opgehaald van Lichess.org:
<https://lichess.org/@/Goran455/download>
- Lichess. (sd). *Lichess open database*. Opgehaald van Lichess.org:
<https://database.lichess.org/>
- Lu, J. (2021, 01 24). *Blogs*. Opgehaald van Chess.com:
<https://www.chess.com/blog/EnergeticHay/25-ways-to-improve-at-chess?>
- Ramponi, J. (2023). *Datasets*. Opgehaald van Kaggle.com:
<https://www.kaggle.com/datasets/jramponi/chess-mapping-file-elo-code-to-opening-name?resource=download>
- Shah, S. (2016, 04 28). *Chess News*. Opgehaald van Chessbase.com:
<https://en.chessbase.com/post/improve-your-chess-with-boris-gelfand-1-2?>
- Silman, J. (2016, 06 09). *Articles*. Opgehaald van Chess.com:
<https://www.chess.com/article/view/longer-time-controls-are-more-instructive?>
- Stockfishchess. (sd). *Download*. Opgehaald van Stockfishchess.org:
<https://stockfishchess.org/download>
- Studer, N. (2021, 10 20). *Articles*. Opgehaald van Nextlevelchess.com:
<https://nextlevelchess.com/games-of-chess-world-champions/>